

Decision Tree

Lecture 2

Part-1



Introduction



Decision trees are supervised, **non-parametric** models used for both classification and regression.



For classification, common split criteria include **Gini impurity** and **entropy/information gain**.



A **non-parametric model** means the model does **not assume a fixed mathematical form or a fixed number of parameters before learning from the data**.



$y = \text{some fixed equation}$

Instead, the tree learns its **structure, splitting features, thresholds, and number of branches** from the training data.

Parametric vs Non-parametric



Parametric	Non-parametric
Assumes a fixed model form	No fixed model form
Fixed number of parameters	Complexity can grow with data
Example: Linear Regression	Example: Decision Tree
Usually simpler	Can model complex patterns
Stronger assumptions	Fewer assumptions about relationship

Example:

Parametric: "I already decided the shape of the model."

Non-parametric: "Learn the shape of the model from the data."

Classification vs Regression Trees

Classification Tree	Regression Tree
Target is categorical For discrete value target functions (labels)	Target is numerical For continuous value target functions (labels)
Example: Yes/No	Example: House price
Gini, Entropy	MSE/variance-based criteria
Output = class	Output = numerical value
Loan approved? → Yes / No Disease? → Positive / Negative Email → Spam / Not Spam	House Price → ₹52 lakh Salary → ₹8.4 lakh Temperature → 31.2°C

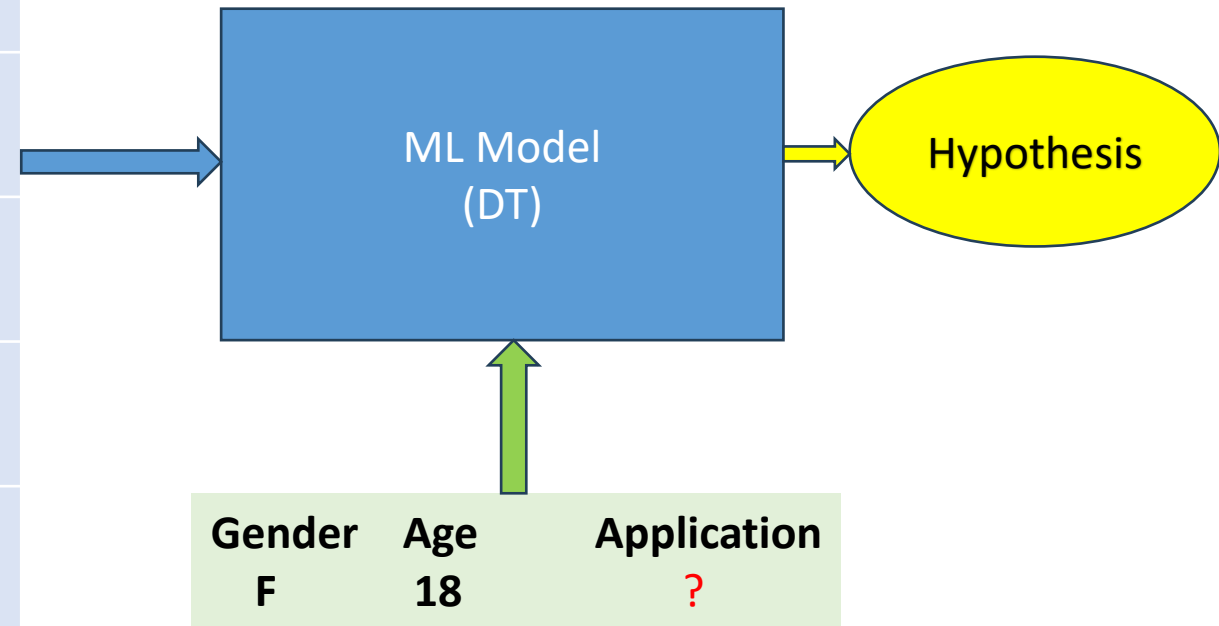
Decision Tree (DT)

- DT breaks down a dataset into **smaller and smaller subsets** while at the same time an **associated decision tree is incrementally developed**.
- For discrete value target functions (for classification)
 - **If-then-else** rule - most widely used approach

Recommending Mobile Applications

Gender	Age	Application
F	15	Pokemon go
F	25	WhatsApp
M	32	Chess
F	40	WhatsApp
M	12	Pokemon go
M	14	Pokemon go

Dataset



Decision Tree - classifier

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

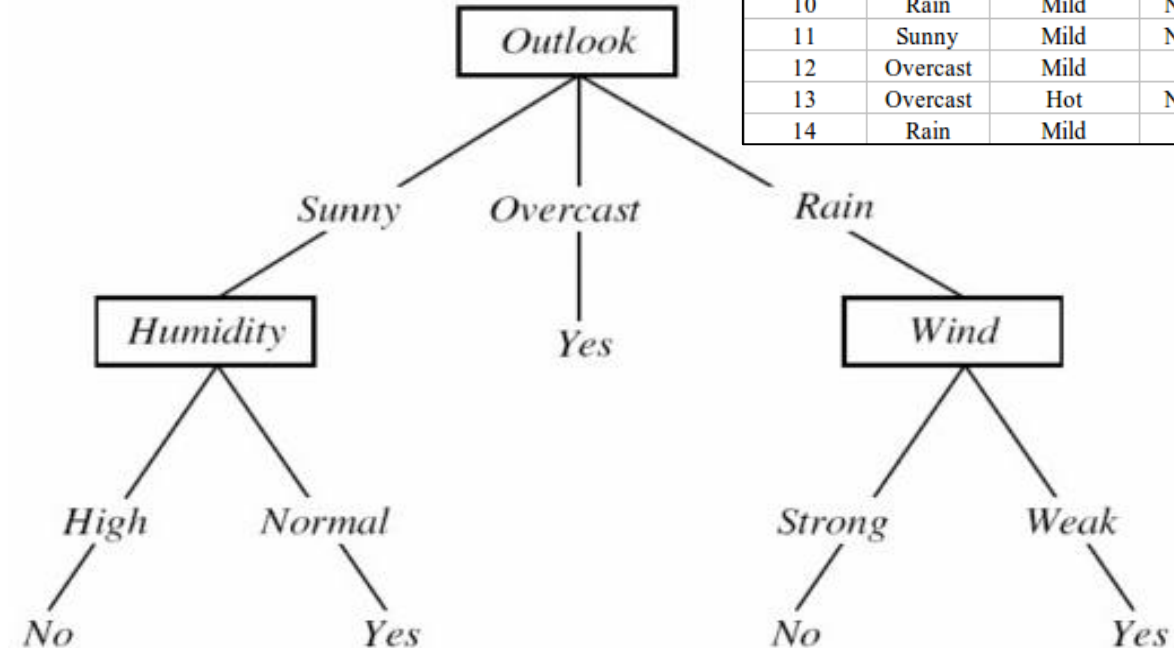
Decision Tree

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Instance classification starts at the **root node** of the tree

Testing the attribute values specified by this node

Moving down the tree branch corresponding to the value of the attribute in the given example.

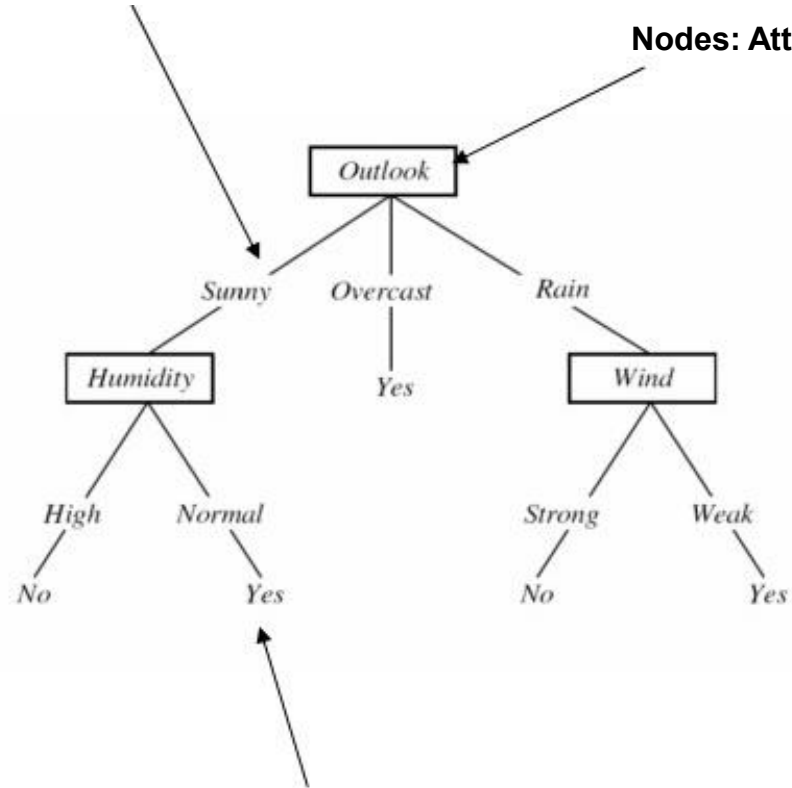


This process is then repeated for the subtree rooted at the new node

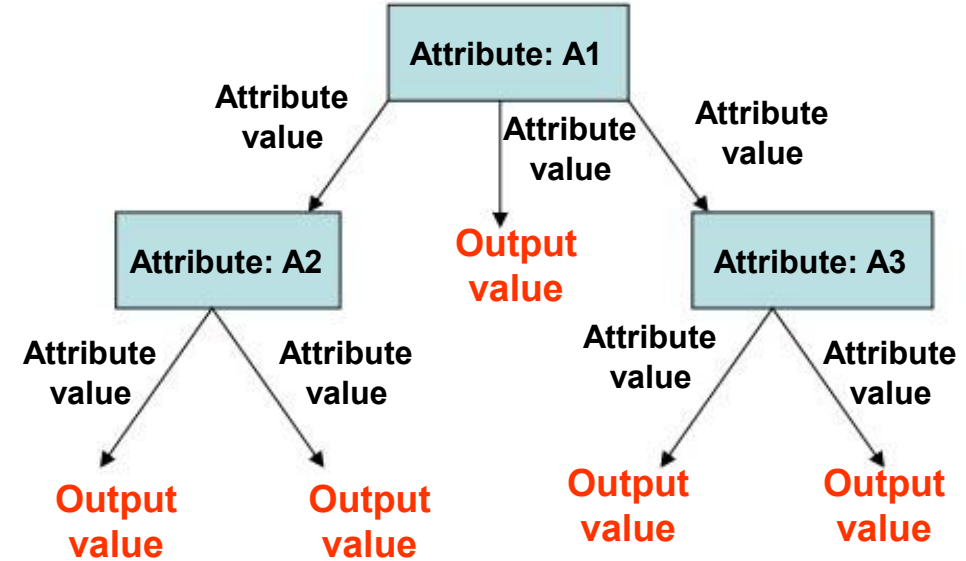
Decision Tree

Edges: Attribute value

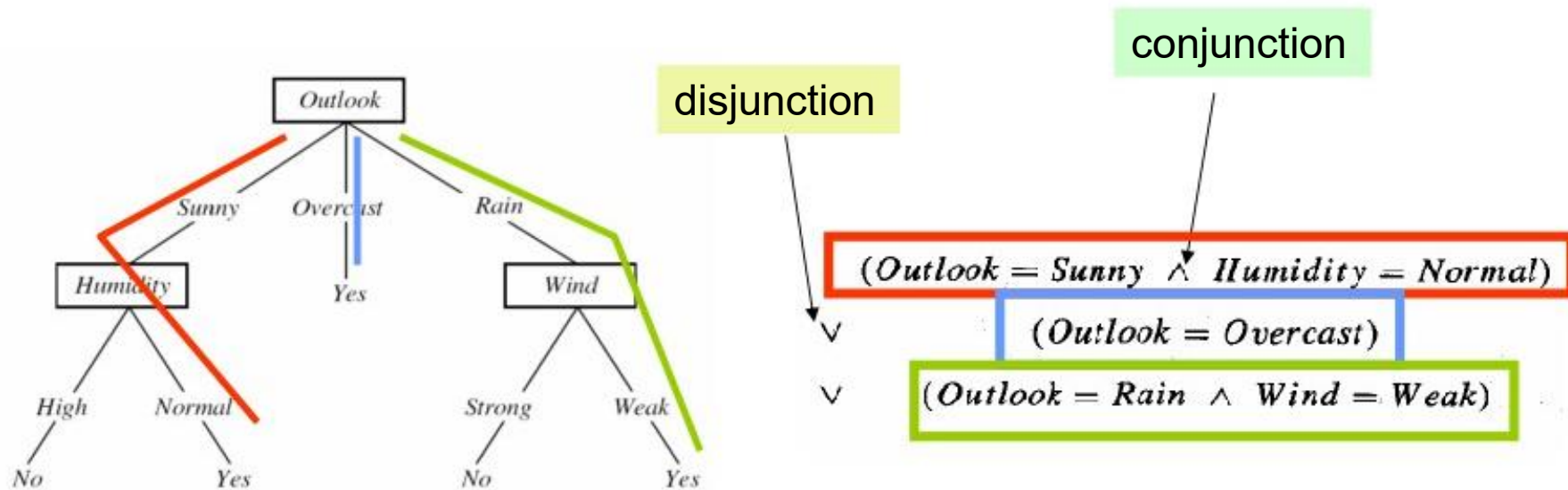
Nodes: Attributes



Leave node:
Output value



Decision Tree

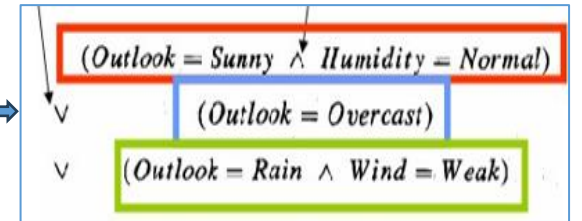


- **Decision trees** represent a disjunction of conjunctions of constraints on the attribute values of instances.
- Each path from the tree root to a leaf corresponds to a conjunction of attribute tests, and the tree itself to a disjunction of these conjunctions.

Decision Tree

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

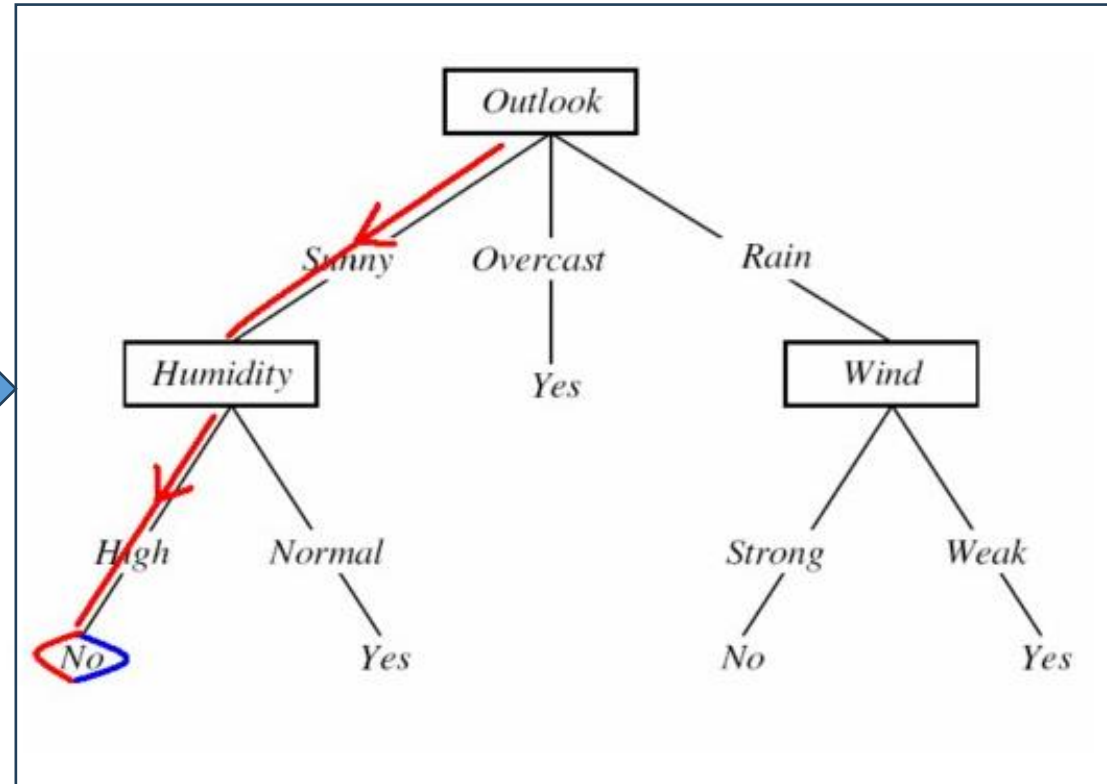
DT classifier



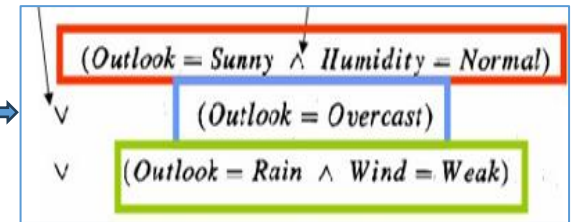
Decision Tree

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Training dataset



Learned concept



<Outlook = Sunny, Temperature = Hot, Humidity = High, Wind = Strong>

Test data

PlayTennis = No

Decision Tree

(a) $A \wedge \neg B$

(b) $A \vee [B \wedge C]$

(c) $A \text{ XOR } B$

(d) $[A \wedge B] \vee [C \wedge D]$

Decision Trees ($F = A \wedge B'$)

If ($A=\text{True}$ and $B = \text{False}$) then Yes

else

No

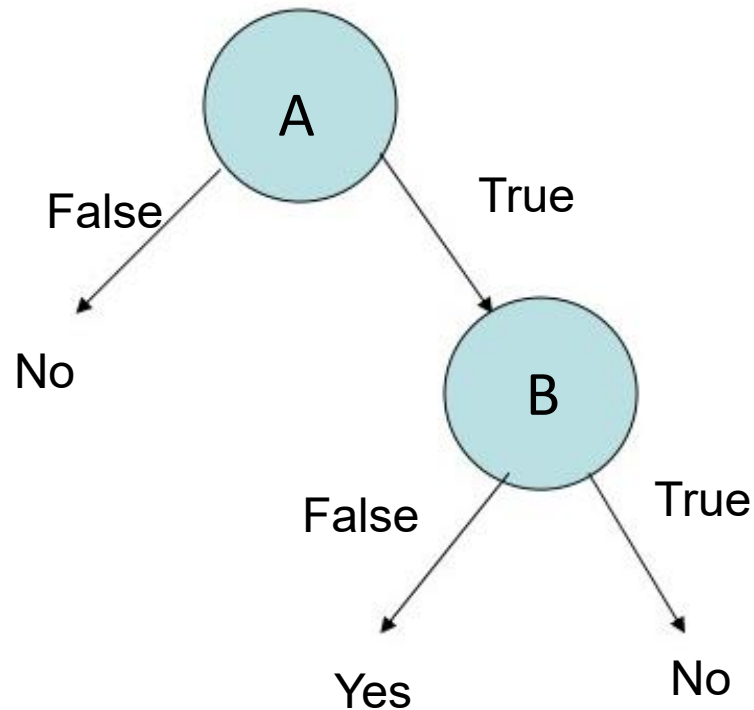
Decision Tree – assignment for IA

- (a) $A \wedge \neg B$
- (b) $A \vee [B \wedge C]$
- (c) $A \text{ XOR } B$
- (d) $[A \wedge B] \vee [C \wedge D]$

Decision Trees ($F = A \wedge B'$)

If ($A=\text{True}$ and $B = \text{False}$) then Yes

else
No



Types of Problems Suitable for Decision Tree Classifier

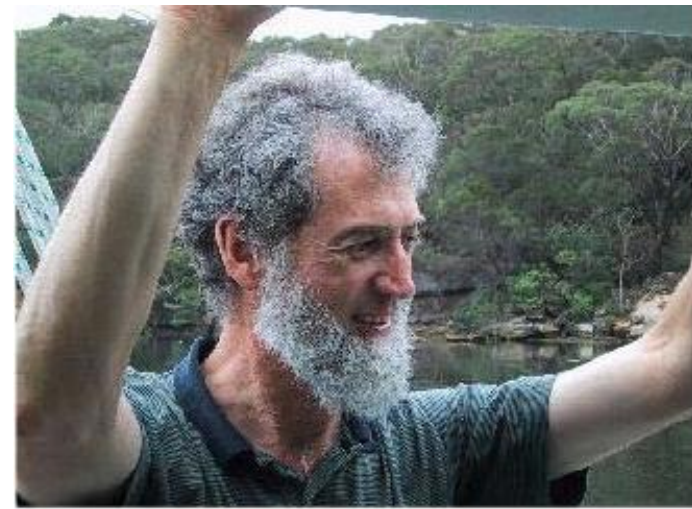
- **Instances** are represented by **multiple attribute values**
 - **Temperature:** (Hot, Mild, Cold)
- The **target function** has **discrete output values**
 - **Play Tennis:** (yes, no)
- **Disjunctive** descriptions may be required
 - $PT = (OL = SU \wedge HU = Nor) \vee (OL = Over) \vee (OL = Ra \wedge Wi = Weak)$
- The training data may contain **errors**
- The training data may contain **missing attribute values**

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Decision Tree Learning

ID3 stands for Iterative Dichotomiser 3

- This algorithm iteratively (**repeatedly**) dichotomizes(**divides**) features into two or more groups at each step.

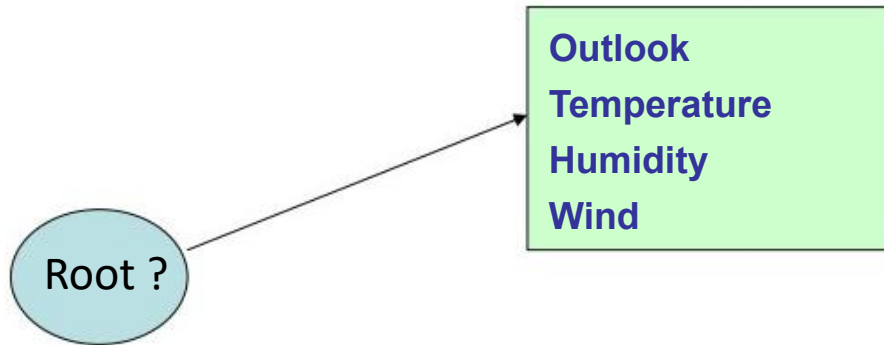


Invented by [Ross Quinlan](#)

ID3 Algorithm (Quinlan 1986) and its successors C4.5 and C5.0

- *Employs a top-down*
- An instance is classified by starting at the **root node** of the tree, testing the attribute specified by this node, then **moving down** the tree branch corresponding to the value of the attribute in the given example.
- This process is then repeated for the **subtree rooted** at the new node.
- Greedy search the space of possible decision trees.
- The algorithm **never backtracks** to reconsider earlier choices.

Which Attribute to Select ??



Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

- Select the attribute that is **most useful** for classifying examples.
- What is a **good quantitative measure** of the worth of an attribute?

ID3 algorithm selecting a best attribute that yields **maximum Information Gain** or **minimum Entropy**.



ID3 Algorithm (Quinlan 1986)

- ID3 uses - **information gain** measure to select among the candidate attributes at each step while growing the tree.
- **Information gain** is based on information theory concept called ***Entropy***
- **Entropy** is a measure of the **uncertainty** associated with a random variable.

Algorithm

1. Calculate entropy for dataset.
2. For each attribute/feature.
 - 2.1. Calculate entropy for all its categorical values.
 - 2.2. Calculate information gain for the feature.
3. Find the feature with maximum information gain.
4. Repeat it until we get the desired tree.

- **Entropy in our Context**

- Given a collection **S**, containing **positive and negative examples** of some target concept, the entropy of S relative to this boolean classification (**yes/ no**) is

$$\text{Entropy (S)} \equiv -(p_{\oplus} \log_2 p_{\oplus} + p_{\ominus} \log_2 p_{\ominus})$$

- Where $p_{\oplus}$ is the proportion of positive examples in **S** and $p_{\ominus}$ is the proportion of negative examples in S.
- In all calculations involving entropy we define **0 log 0** to be **0**.

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Entropy

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

- There are 14 examples. **9 positive** and **5 negative** examples [9+, 5-].
- The entropy of S relative to this Boolean (yes/no) classification is

$$\begin{aligned} \text{Entropy}([9+, 5-]) &= -(9/14) \log_2(9/14) - (5/14) \log_2(5/14) \\ &= 0.940 \end{aligned}$$

Information Gain Measure

Information gain, is simply the expected reduction in entropy caused by partitioning the examples according to this attribute.

More precisely, the information gain, **$Gain(S, A)$** of **an** attribute **A** , relative to a collection of examples **S** , is defined as

$$Gain(S, A) \equiv Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

where **$Values(A)$** is the set of all possible values for attribute **A** , and **S_v** , is the subset of **S** for which attribute **A** has value **v** , i.e.,

$$S_v = \{s \in S | A(s) = v\}$$

Gain (S, outlook)

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Gain (S, Temperature)

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Gain (S, Humidity)

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Gain (S, Wind)

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Gain (S,A)

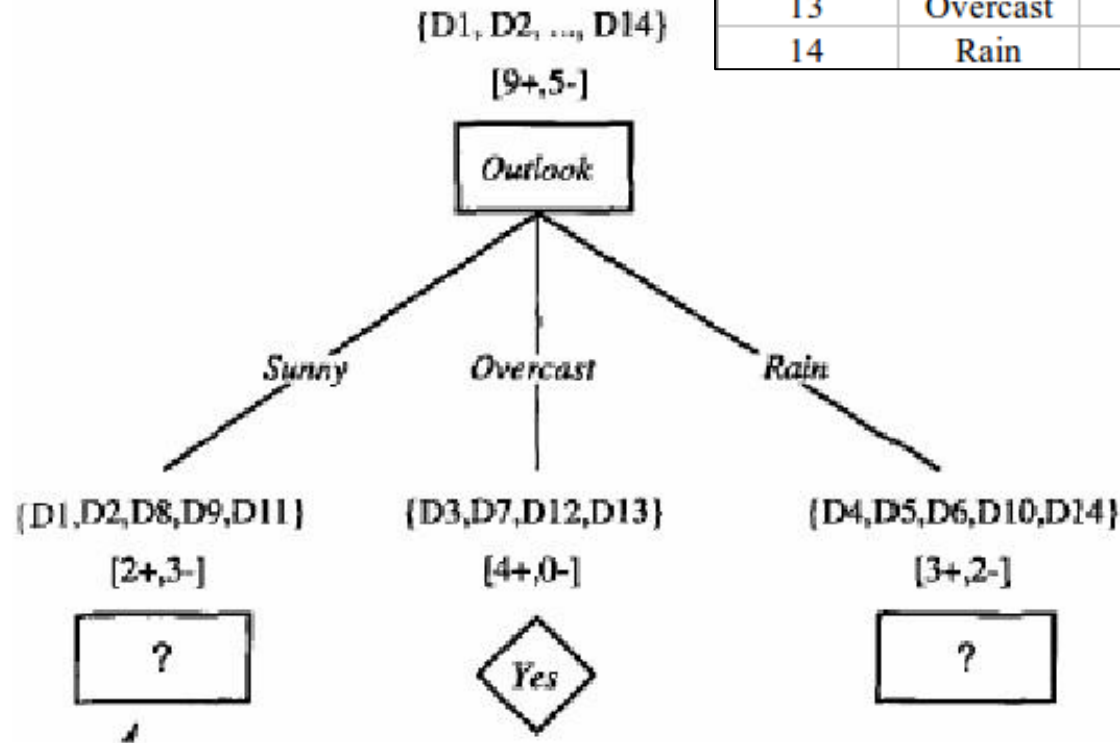
$$\underline{Gain(S, Outlook) = 0.246}$$

$$Gain(S, Humidity) = 0.151$$

$$Gain(S, Wind) = 0.048$$

$$Gain(S, Temperature) = 0.029$$

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No



Gain (S, Outlook) → Higher

Outlook is selected as root

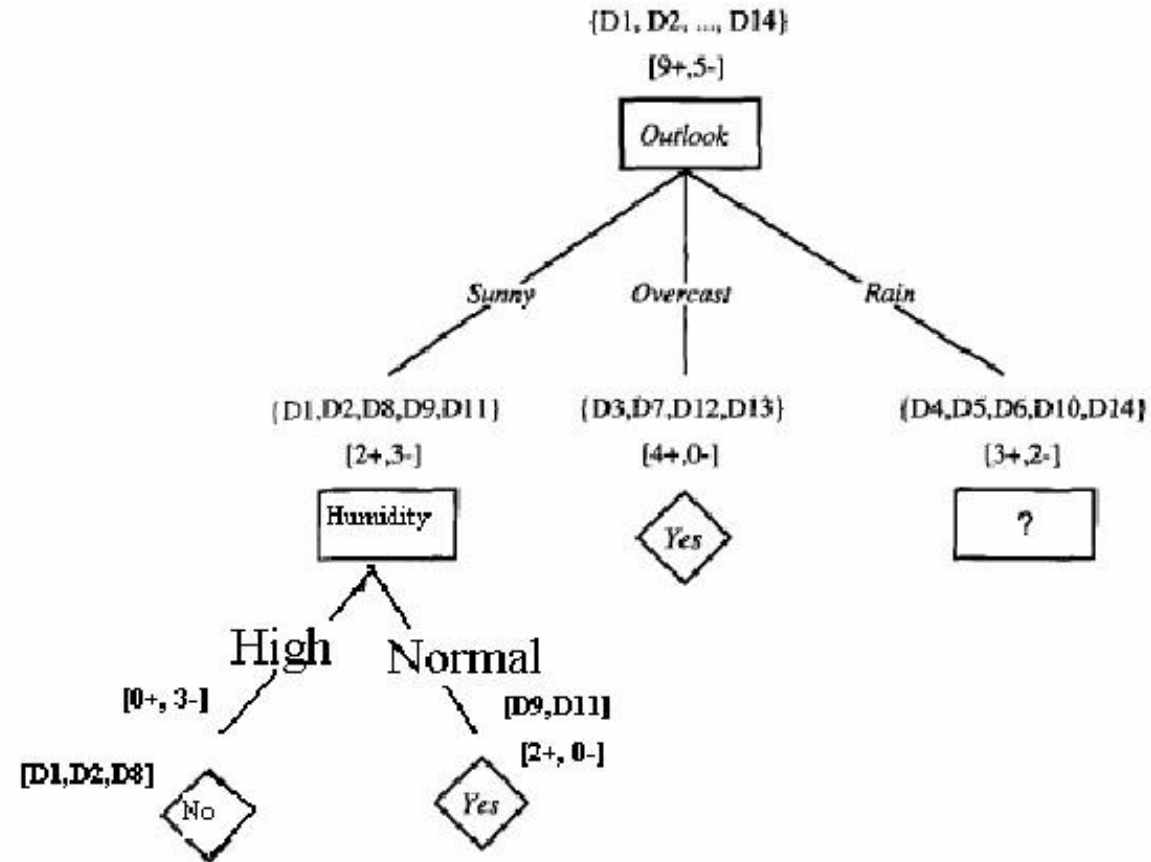
Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Gain (Sunny, A)

A = Temperature, Humidity, Wind

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

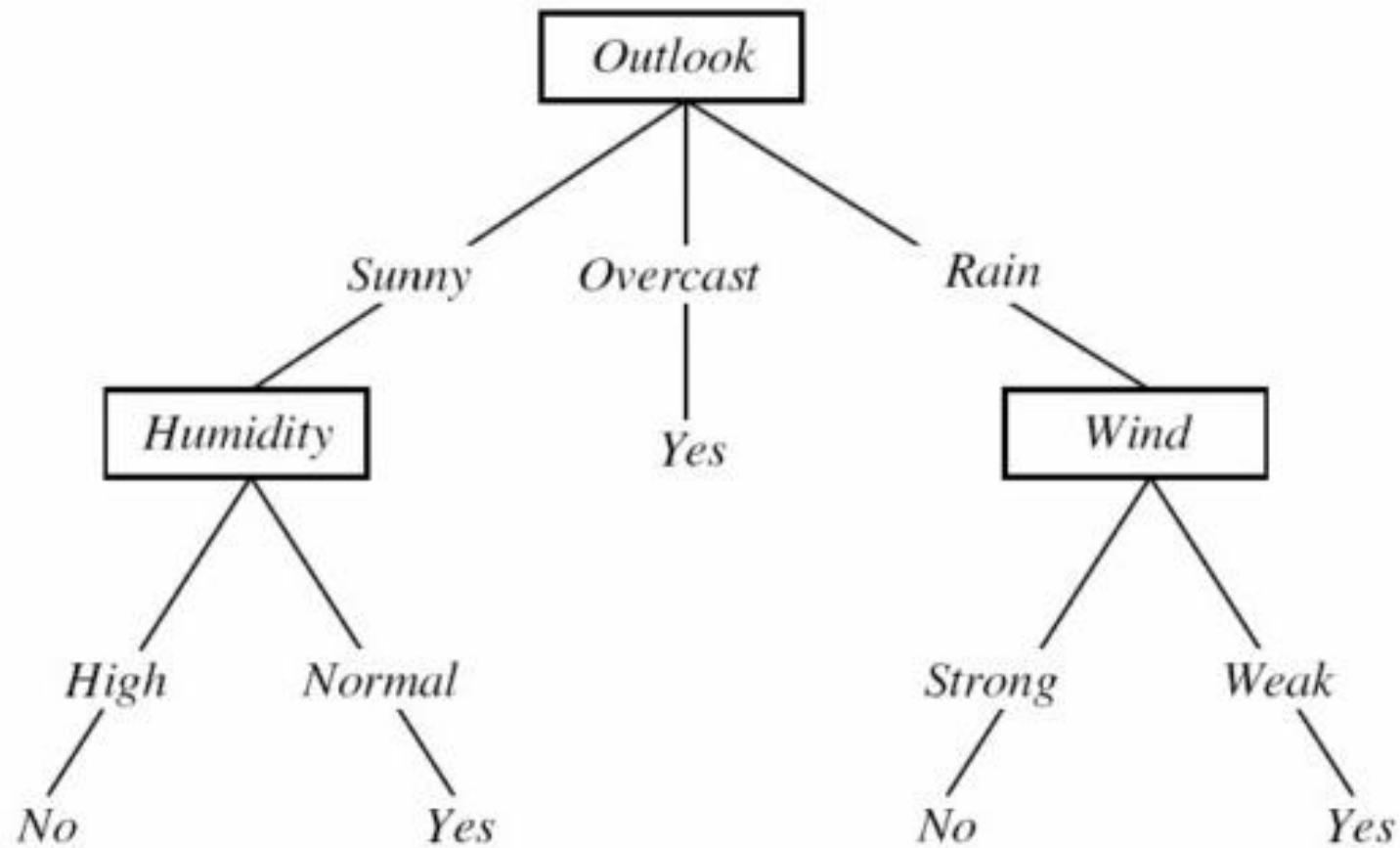
Modified Decision Tree



Gain (Rain, A)

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Final Decision Tree



Whether Dom will play tennis or not on D15?

	OL	Temp	Hum	Wind
D15	Rain	Mild	High	Weak ?

Assignment: Apply ID3 algorithm to obtain a DT for the following data

Instance	Attribute_1	Attribute_2	Class
1	T	T	+
2	T	T	+
3	T	F	-
4	F	F	+
5	F	T	-
6	F	T	-

Capabilities and Limitations of ID3 Algorithm

Advantage:

- ID3 algorithm searches complete hypothesis space
- ID3 uses all training examples at each step in the search.
- The resulting search is much less sensitive to errors in individual training examples.

Capabilities and Limitations of ID3 Algorithm

- Disadvantage:
 - ID3 maintain only a single current hypothesis as it searches through the space of decision trees. Loses the capabilities of representing all consistent hypothesis.
 - ID3 performs no backtracking in its search.
 - Once it selects an attribute to test at a particular level in the tree, it never backtracks to reconsider this choice.
 - Therefore, converging to locally optimal solutions that are not globally optimal.

Decision Tree Approach-2

Measure of impurity: Gini Index

Gini Index

Gini index or Gini impurity measures the **degree or probability** of a particular variable being **wrongly classified when it is randomly chosen**.

- It means an attribute **with lower Gini index** should be preferred.
- The degree of Gini index varies between **0 and 1**.
 - **0 denotes** that all elements belong to a certain class or if there exists only **one class**
 - **1 denotes** that the elements are **randomly distributed across various classes**.
- A Gini Index of **0.5** denotes **equally distributed** elements **into some classes**.

Gini Index

- The Formula for the calculation of the Gini Index is given below.

$$Gini = 1 - \sum_{i=1}^C (p_i)^2$$

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Gini Index

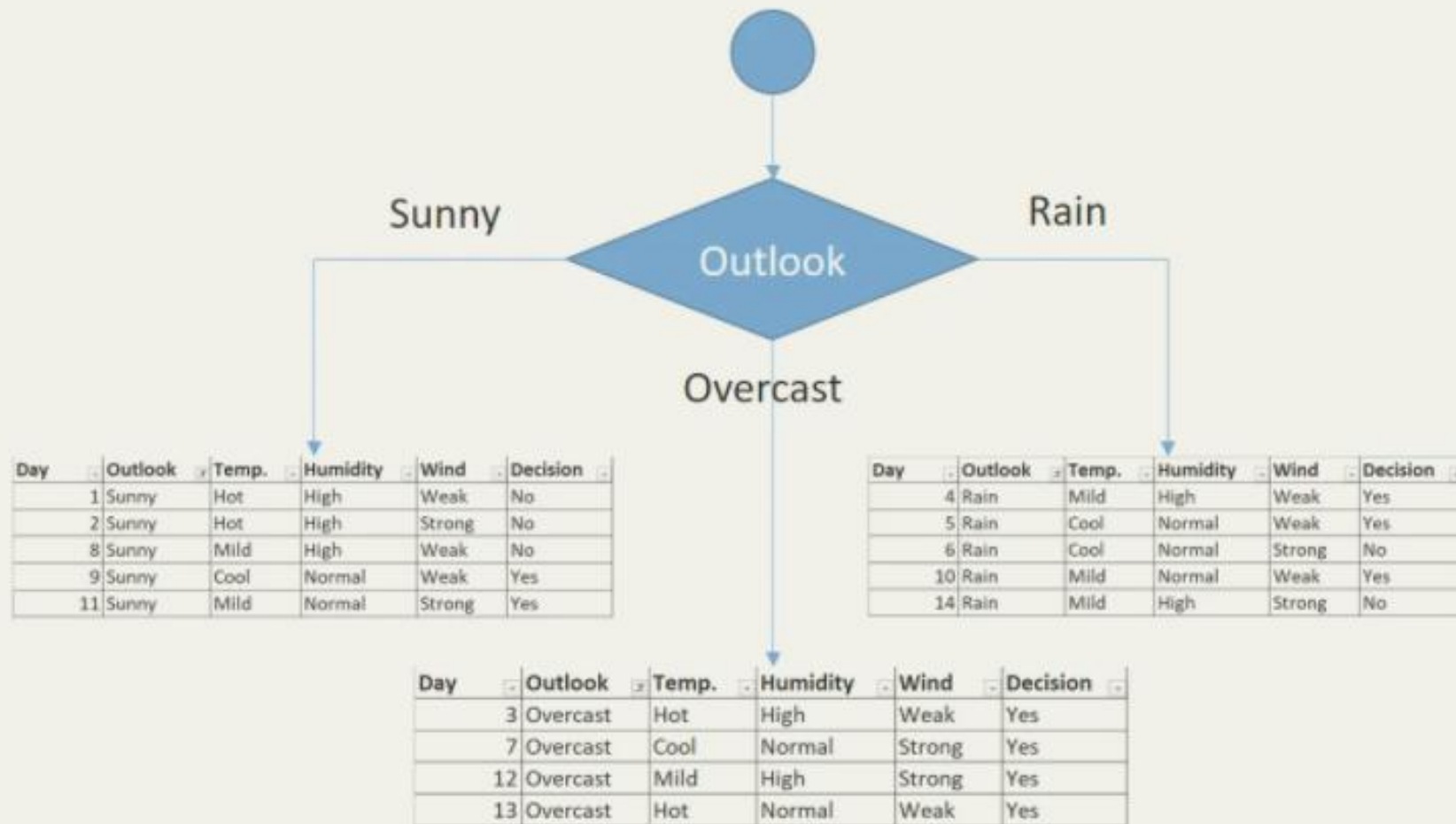
- The Formula for the calculation of the Gini Index is given below.

$$Gini = 1 - \sum_{i=1}^C (p_i)^2$$

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Gini Index

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No



First decision would be outlook feature

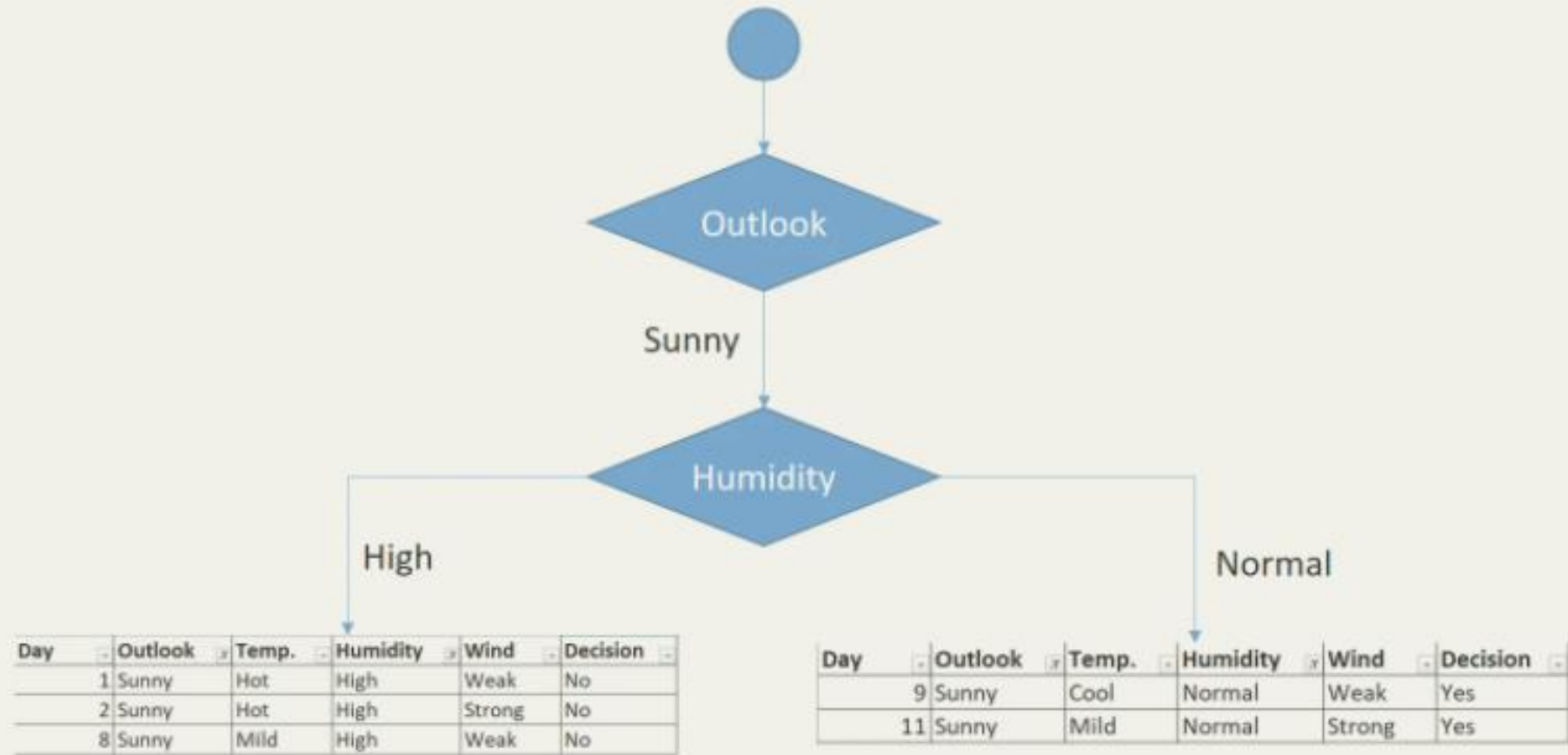
You might realize that sub dataset in the overcast leaf has only yes decisions. This means that overcast leaf is over.



Tree is over for overcast outlook leaf

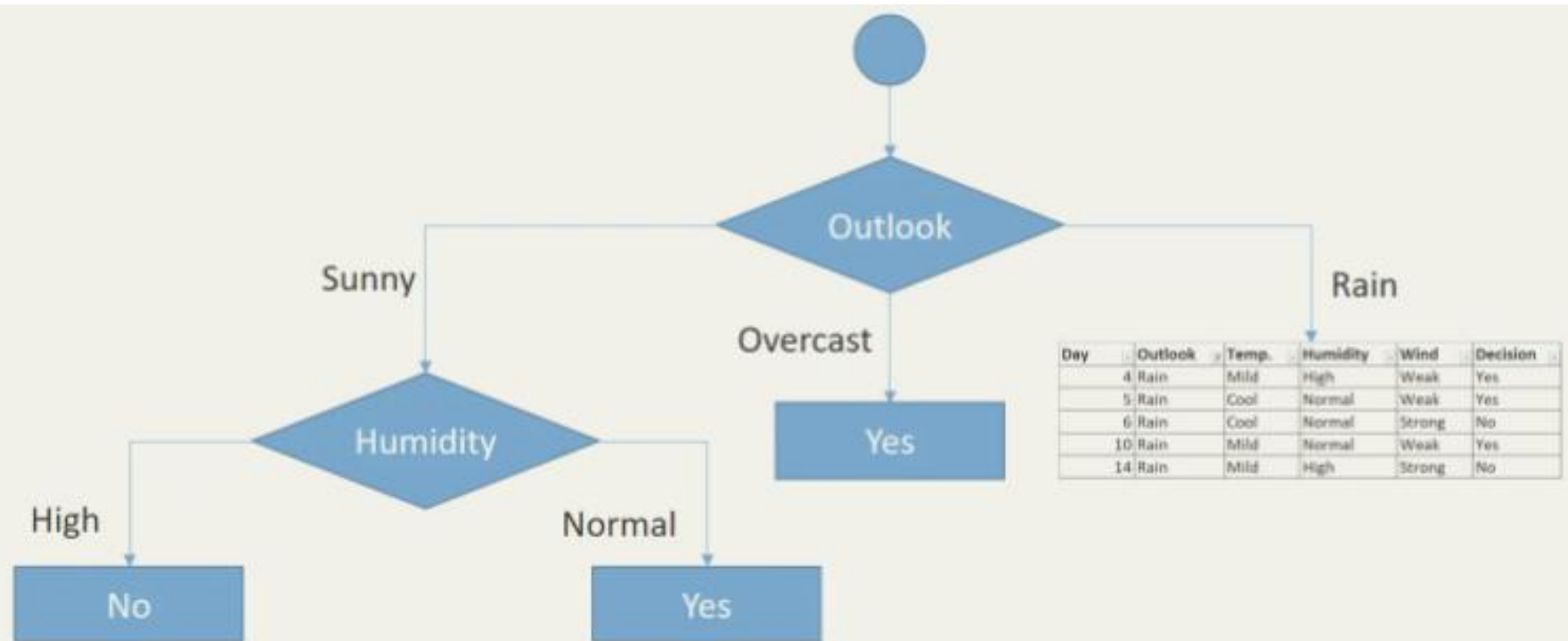
We will apply same principles to those sub datasets in the following steps.

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No



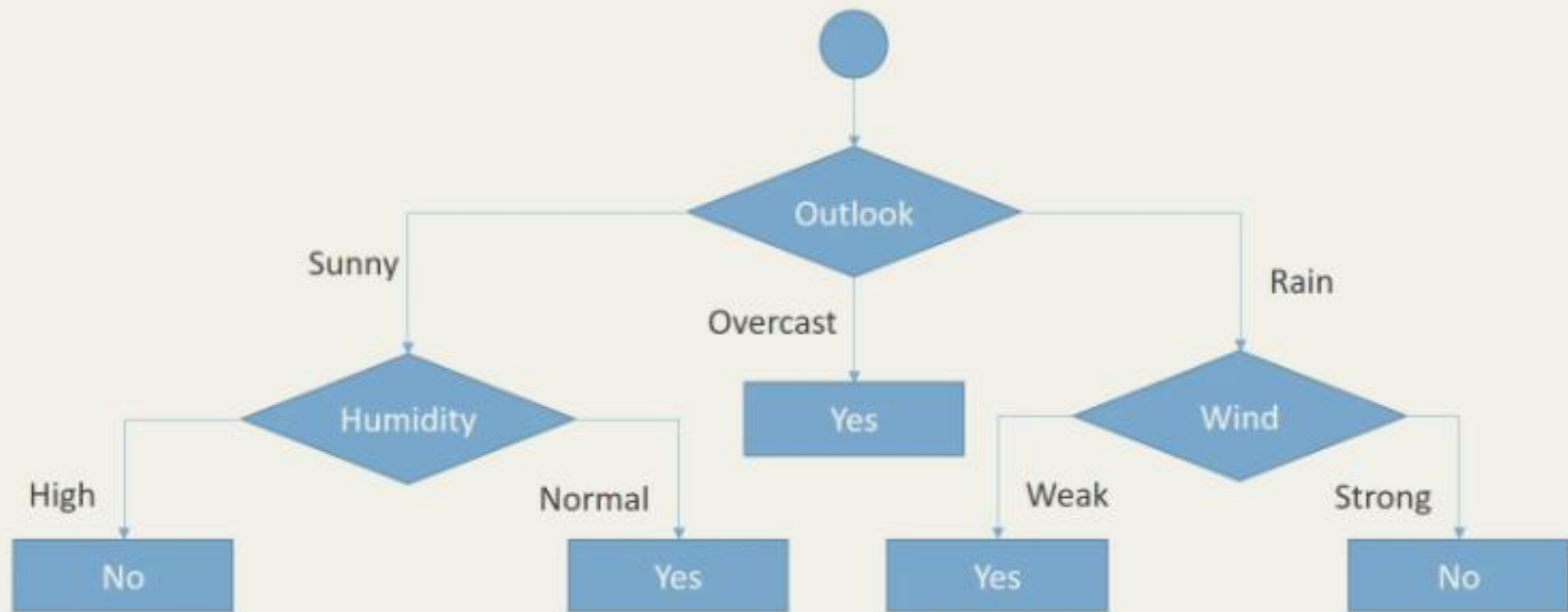
Sub datasets for high and normal humidity

As seen, decision is always no for high humidity and sunny outlook. On the other hand, decision will always be yes for normal humidity and sunny outlook. This branch is over.



Decisions for high and normal humidity

Now, we need to focus on rain outlook.



Final form of the decision tree built by CART algorithm

Training and Testing

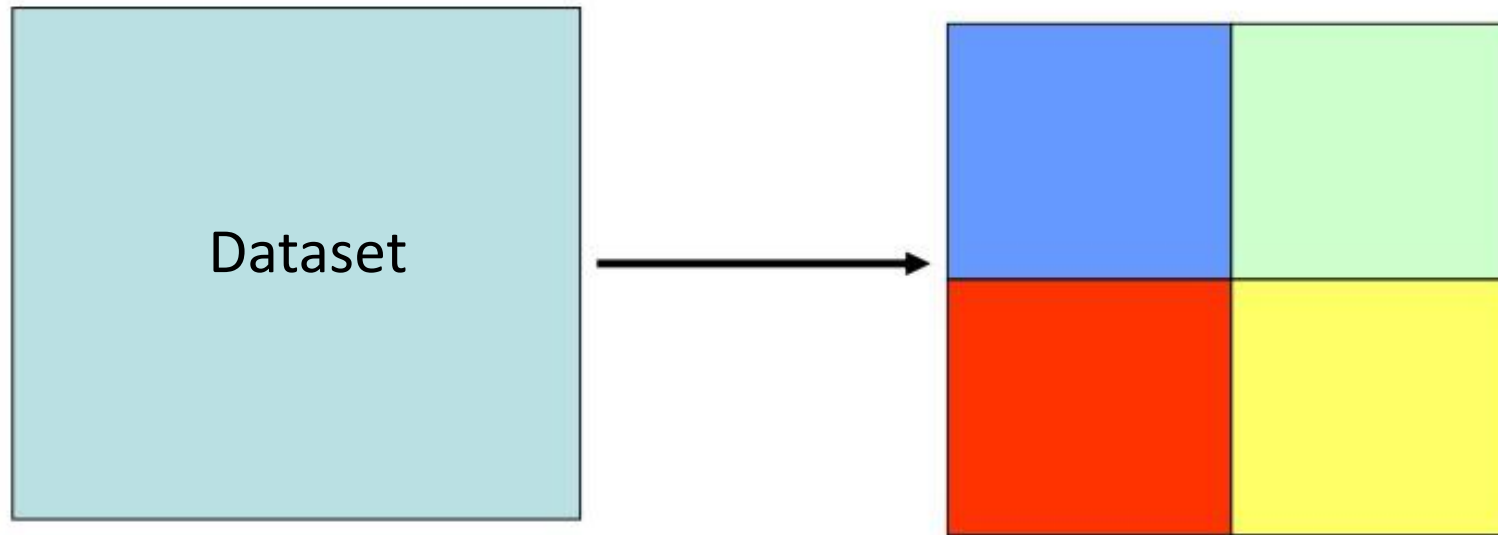
Training and Testing

	Day	Outlook	Temperature	Humidity	Wind	Play Tennis
Training set	1	Sunny	Hot	High	Weak	No
	2	Sunny	Hot	High	Strong	No
	3	Overcast	Hot	High	Weak	Yes
	4	Rain	Mild	High	Weak	Yes
	5	Rain	Cool	Normal	Weak	Yes
	6	Rain	Cool	Normal	Strong	No
	7	Overcast	Cool	Normal	Strong	Yes
	8	Sunny	Mild	High	Weak	No
	9	Sunny	Cool	Normal	Weak	Yes
	10	Rain	Mild	Normal	Weak	Yes
Validation	11	Sunny	Mild	Normal	Strong	Yes
	12	Overcast	Mild	High	Strong	Yes
Test	13	Overcast	Hot	Normal	Weak	??
	14	Rain	Mild	High	Strong	

K-fold Cross validation technique

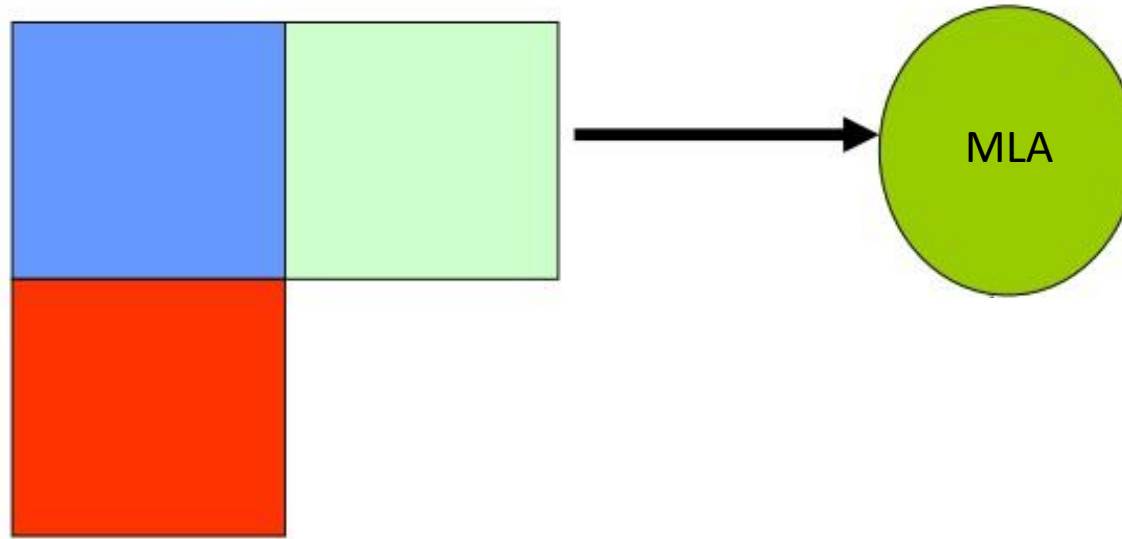
- Cross-validation is a **statistical method** used to estimate the ability of machine learning models.
- It is commonly used to compare and select a model.
- The parameter k refers to the number of **groups** that a given data sample is to be split into.
- The procedure is often called k-fold cross-validation.
- When $k=4$ becoming 4-fold cross-validation

4-Fold Cross-validation



Dataset is partitioned randomly into 4 equal sets

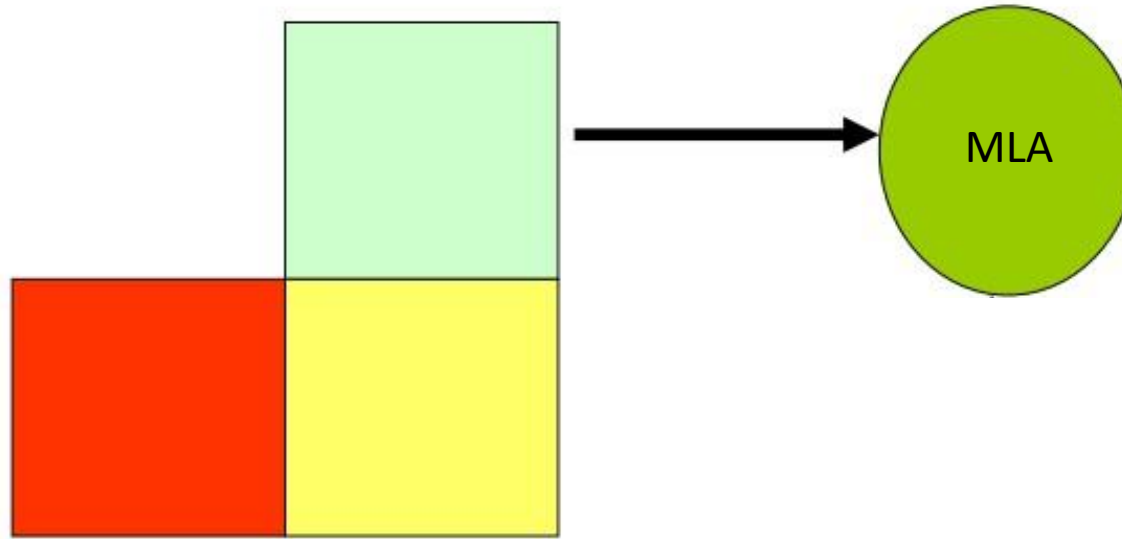
4-Fold Cross-validation



Training Dataset

Training and validation of each classifier were carried out 4 times using one distinct set for testing and other 4-1 sets for training.

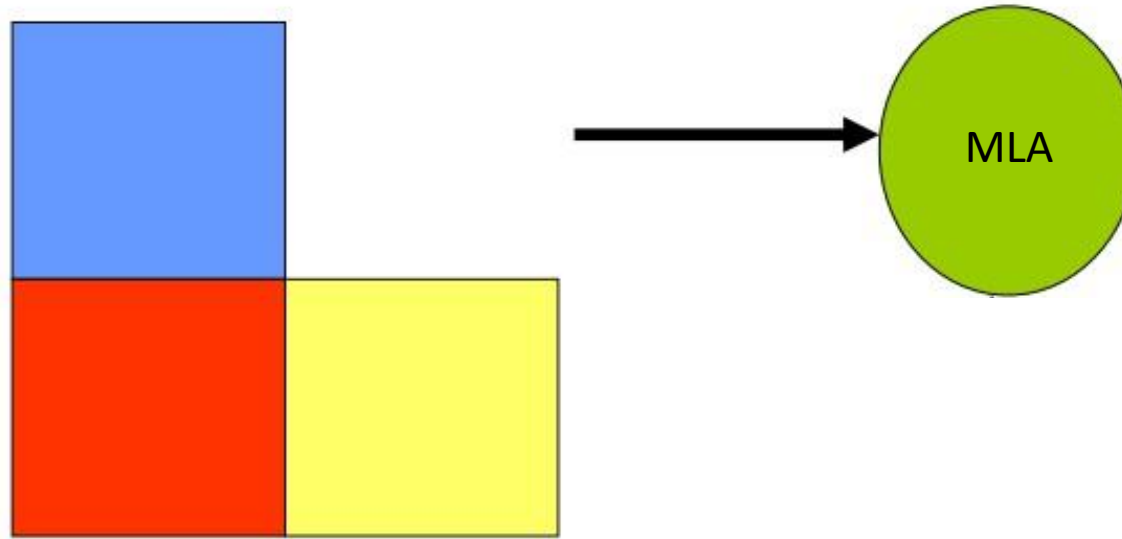
4-Fold Cross-validation



Training Dataset

Training and validation of each classifier were carried out 4 times using one distinct set for testing and other 4-1 sets for training.

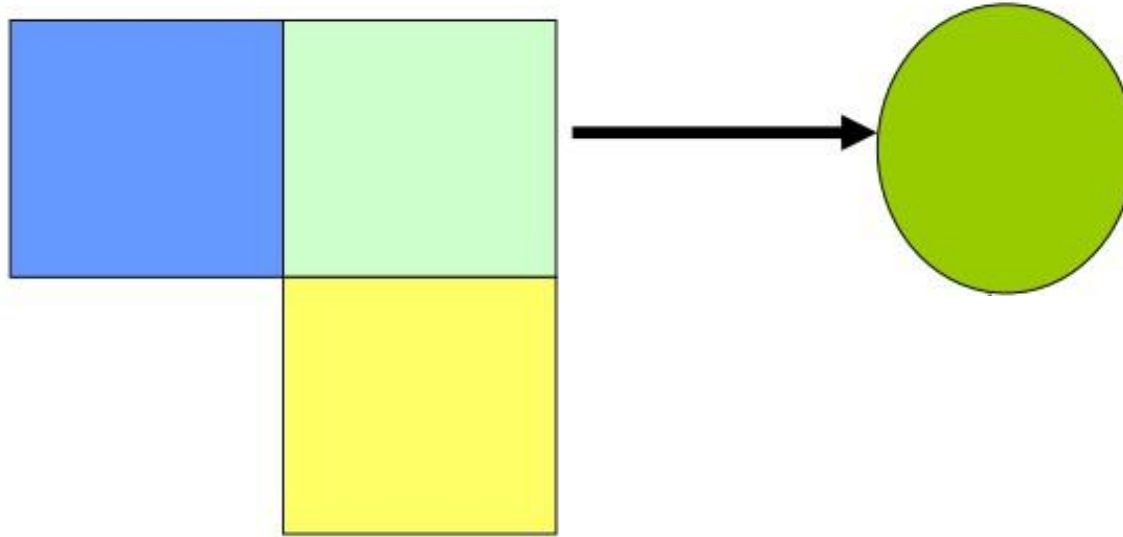
4-Fold Cross-validation



Training Dataset

Training and validation of each classifier were carried out 4 times using one distinct set for testing and other 4-1 sets for training.

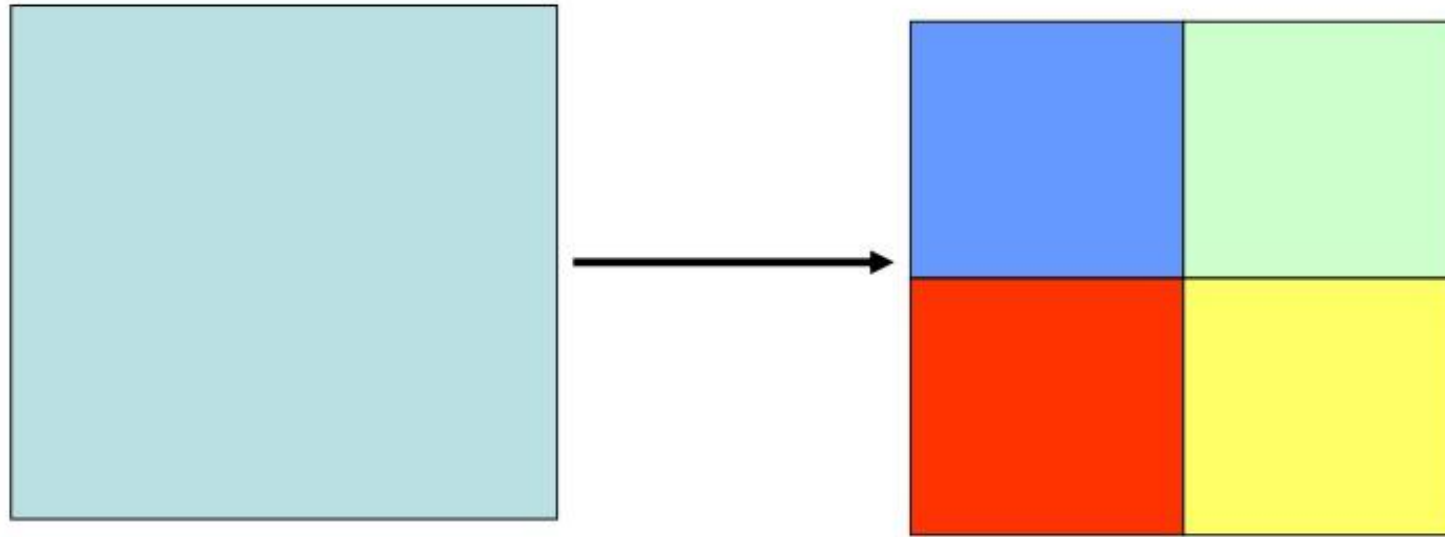
4-Fold Cross-validation



Training Dataset

Training and validation of each classifier were carried out 4 times using one distinct set for testing and other 4-1 sets for training.

4-Fold Cross-validation



$$\text{ACC} = (\text{ACC1} + \text{ACC2} + \text{ACC3} + \text{ACC4}) / 4$$

Training and validation of each classifier were carried out 4 times using one distinct set for testing and other 4-1 sets for training.

Selection or Configuration of k

- The k value must be chosen carefully for your data sample.
- A **poorly chosen value for k** may result in a misrepresentative idea of the skill of the model, such as a score with a **high variance** (that may change a lot based on the data used to fit the model), or a **high bias**, (such as an overestimate of the skill of the model).
- Three common tactics for choosing a value for k are as follows:
 - **Representative**
 - **K=5 or 10**
 - **K=n**

Example

Data sample with 6 observations [0.1, 0.2, 0.3, 0.4, 0.5, 0.6]

Select a value for k to determine the number of folds used to split the data

- we will shuffle the data and then split the data into 3 groups.
- $k=3$
- we have 6 observations; each group will have an equal number of 2 observations.

G1: [0.5, 0.2]

G2: [0.1, 0.3]

G3: [0.4, 0.6]

Example

Data sample with 6 observations [0.1, 0.2, 0.3, 0.4, 0.5, 0.6]

Select a value for k to determine the number of folds used to split the data

- we will shuffle the data and then split the data into 3 groups.
- $k=3$
- we have 6 observations; each group will have an equal number of 2 observations.

G1: [0.5, 0.2]

G2: [0.1, 0.3]

G3: [0.4, 0.6]

- **Model1: Trained on G1 + G2, Tested on G3**
- **Model2: Trained on G2 + G3, Tested on G1**
- **Model3: Trained on G1 + G3, Tested on G2**

The models are then discarded after they are evaluated as they have served their purpose.

Cross-Validation API

The **scikit-learn library** provides an implementation that will split a given data sample up

KFold (*number of splits*, *whether or not to shuffle the sample*, *seed* for the [pseudorandom number generator](#) used prior to the shuffle) scikit-learn class can be used.

KFold (3 folds, shuffles prior to the split, uses a value of 1 for the pseudorandom number generator).

```
kfold = KFold(3, True, 1)
```

```
# split function
```

```
for train, test in kfold.split(data):
```

```
    print('train: %s, test: %s' % (train, test))
```

- Model1: Trained on G1 + G2, Tested on G3
- Model2: Trained on G2 + G3, Tested on G1
- Model3: Trained on G1 + G3, Tested on G2

Issues in Decision Tree Learning

- Determining how deeply to grow the decision tree
- Handling continuous attributes
- Choosing an appropriate attribute selection measure
- Handling training data with missing attribute values
- Handling attributes with differing costs

Issues in Decision Tree Learning

- Determining **how deeply to grow** the decision tree
- Handling continuous attributes
- Choosing an appropriate attribute selection measure
- Handling training data with missing attribute values
- Handling attributes with differing costs

(1) Determining **how deeply to grow** the decision tree

Overfitting

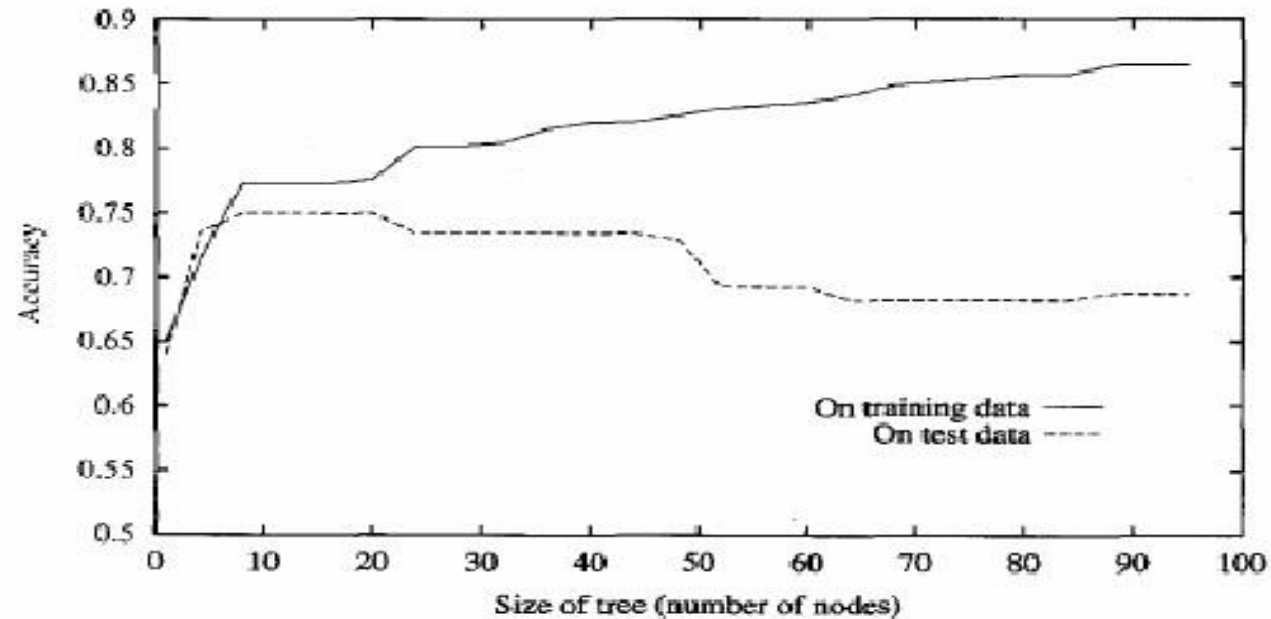
- **Training**: Task of fitting the **model/ algorithm** to a set of **training** data, in order to make the reliable prediction on **unseen test** data

ID3 algorithm can produce trees that **overfit** the training examples

- When there is **noise** in the data or
- When the number of training examples is **too small**.

(1) Determining **how deeply to grow** the decision tree

Overfitting



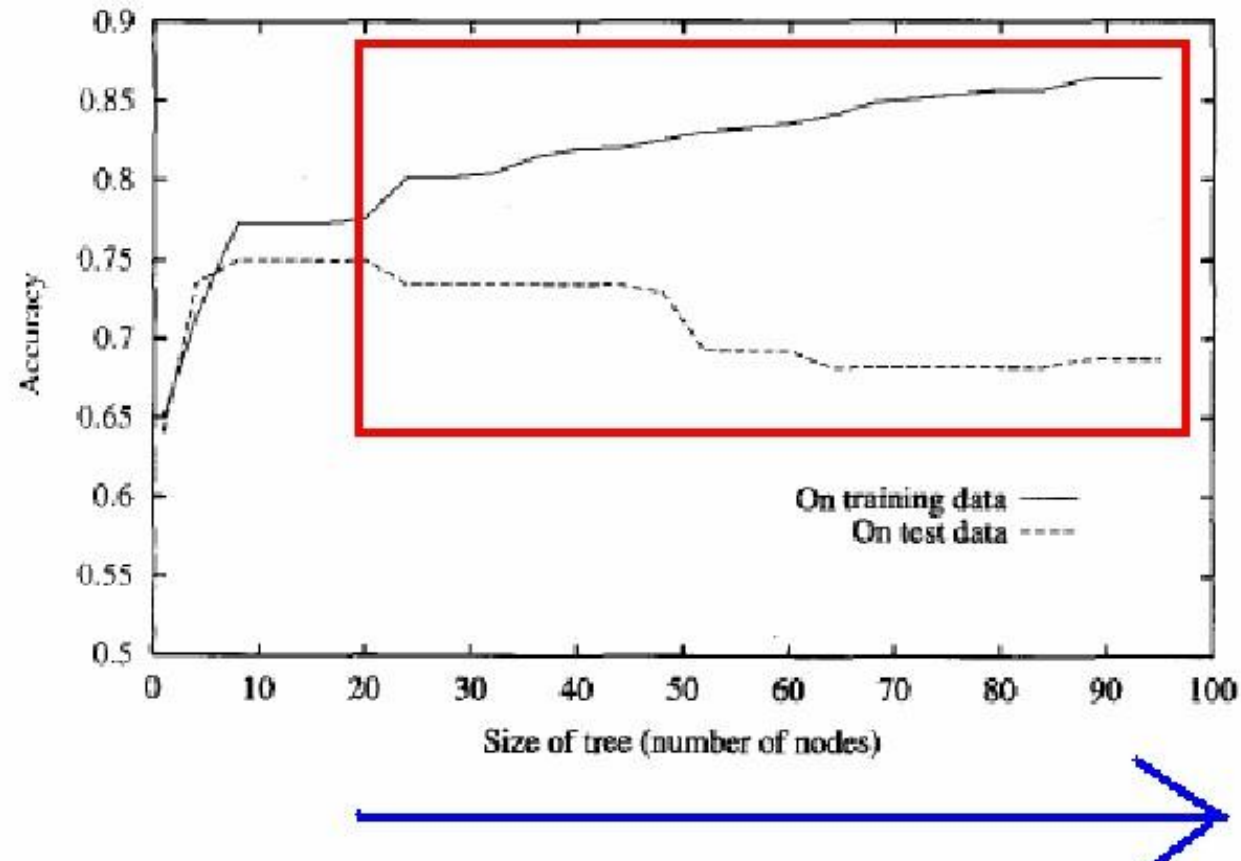
ID3 algorithm is applied to the task of learning which medical patients have a form of diabetes.

The accuracy of the tree over the training examples increases monotonically as the tree is grown.

However, the accuracy measured over the independent test examples first increases, then decreases

(1) Determining **how deeply to grow** the decision tree

Overfitting in Decision Tree



Overfitting in decision tree learning. As ID3 adds new nodes to grow the decision tree, the accuracy of the tree measured over the training examples increases monotonically. However, when measured over a set of test examples independent of the training examples, accuracy first increases, then decreases.

(1) Determining **how deeply to grow** the decision tree

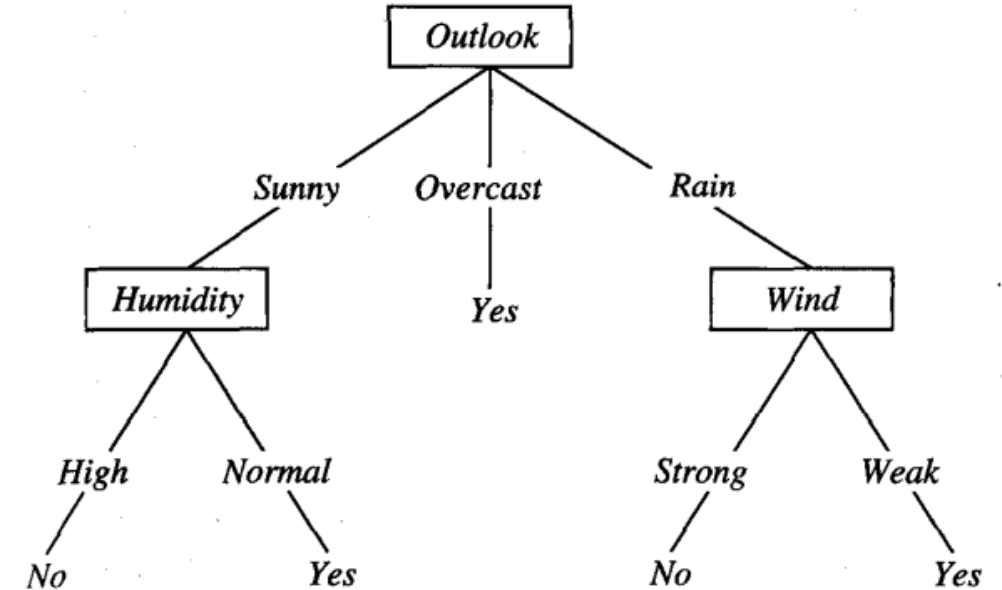
Why Overfitting Happens in Decision Tree Learning?

- **Presence of error in the training examples. (common in machine learning)**
- **When small numbers of examples** are associated with leaf nodes.

(1) Determining **how deeply to grow** the decision tree

Presence of Error and Over-fitting

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No



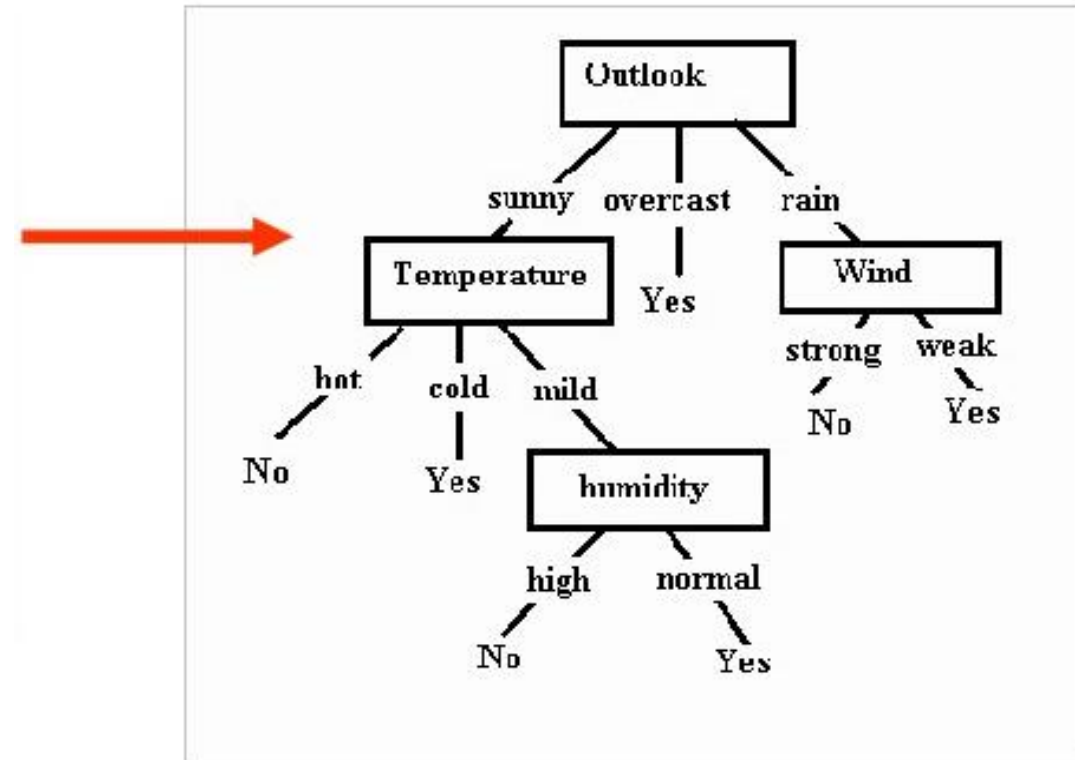
(D15 Outlook = Sunny, Temperature = Hot, Humidity = Normal, Wind = Strong, PlayTennis = Yes) ✓

(D15 Outlook = Sunny, Temperature = Hot, Humidity = Normal, Wind = Strong, PlayTennis = No) ✗

Adding the following positive training example, **incorrectly labeled as negative**

Presence of Error and Over-fitting

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No



(D15 Outlook = Sunny, Temperature = Hot, Humidity = Normal, Wind = Strong, PlayTennis = No)

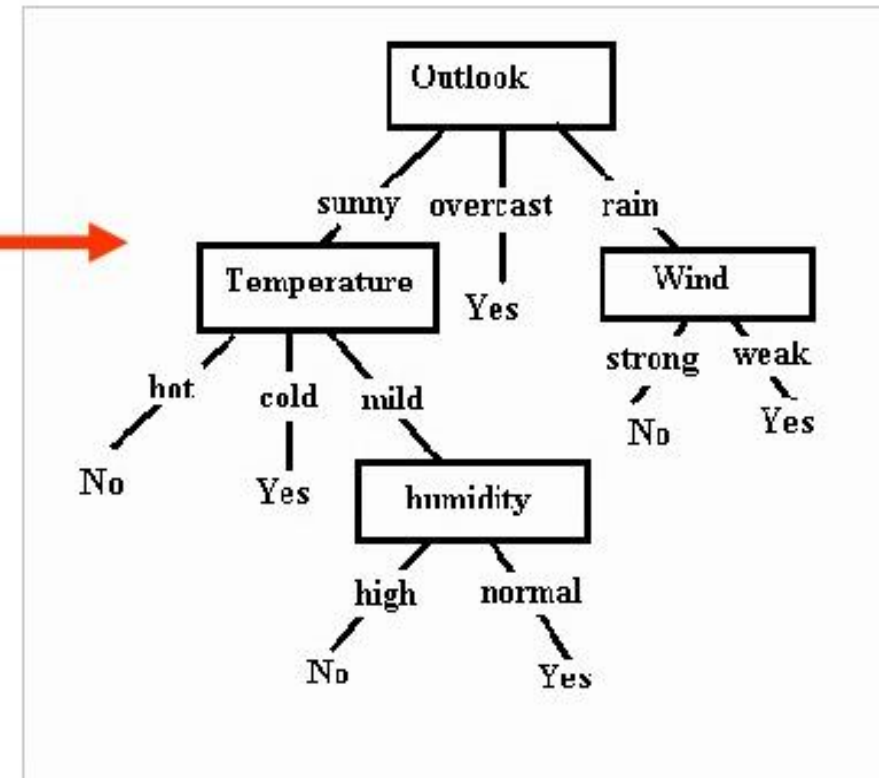
Adding the following positive training example, **incorrectly labeled as negative**

Presence of Error and Over-fitting

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

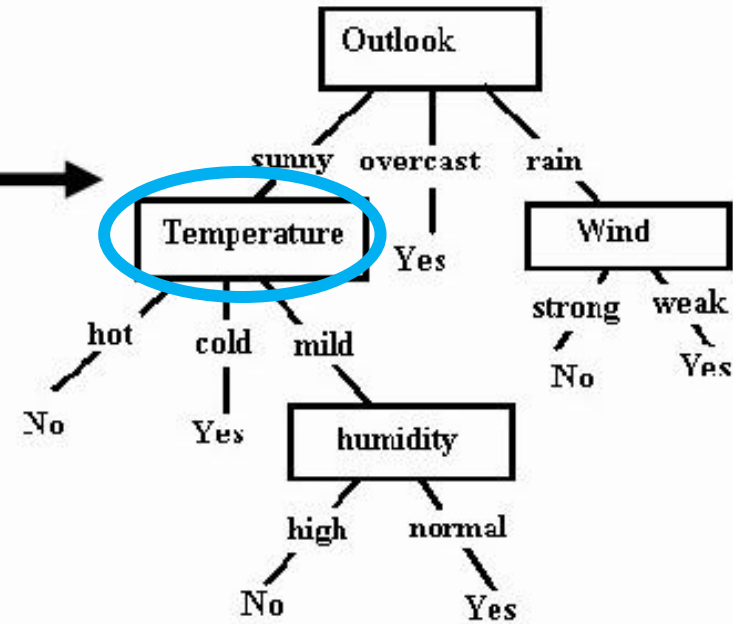
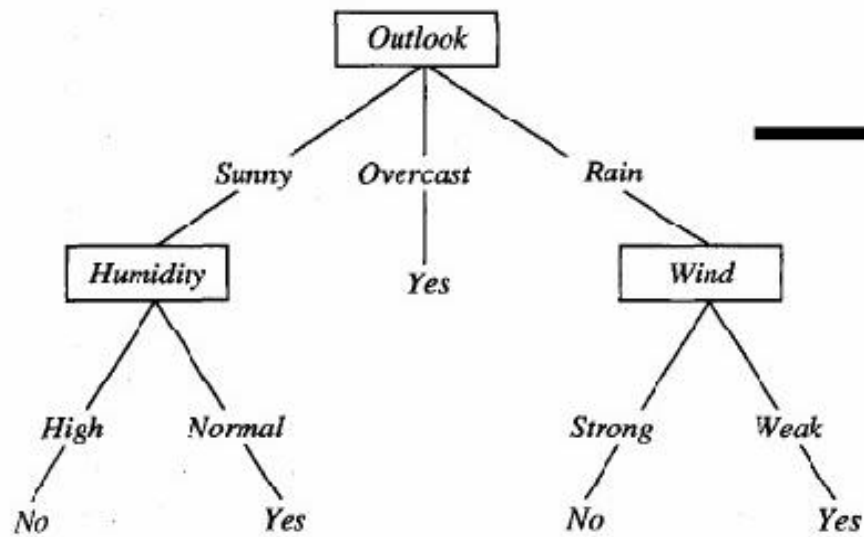
+

*(Outlook = Sunny, Temperature = Hot, Humidity = Normal,
Wind = Strong, PlayTennis = No)*



This example illustrates how random noise in the training examples can lead to overfitting.

Presence of Error and Over-fitting



Whether Tom will play tennis or not on D16?

OL Tem Hum Wind

D16 Sunny Mild High Weak ?

**More Complex
Tree depth is more**

Why to avoid Overfitting ??

Experimental study of ID3 with **noisy data** → **overfitting**

Decrease the accuracy of decision trees by **10-25%** on most problems.

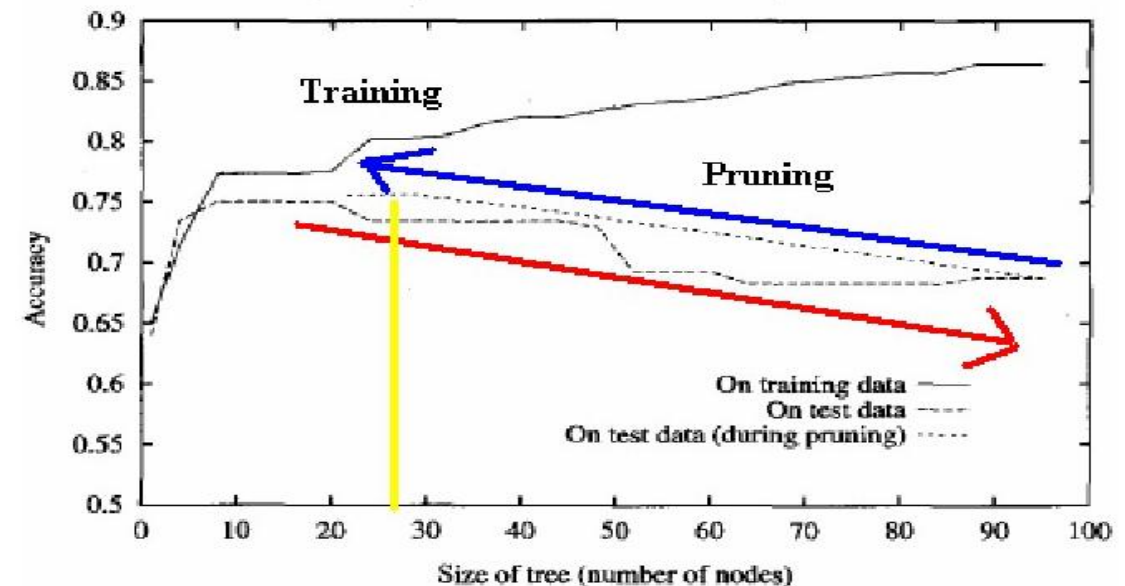
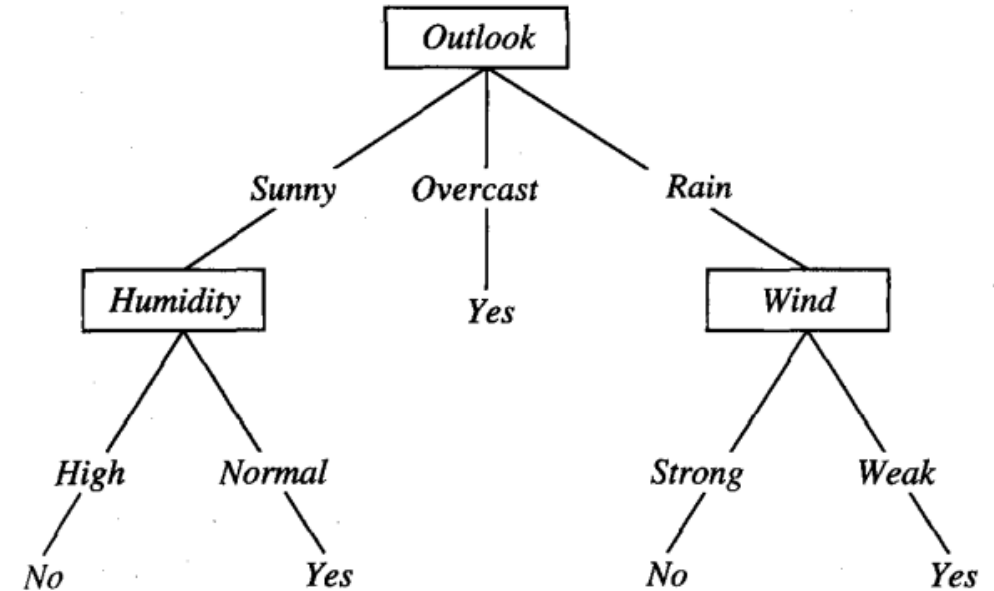
How to avoid Overfitting

There are several approaches to avoiding overfitting in decision tree learning. These can be grouped into two classes:

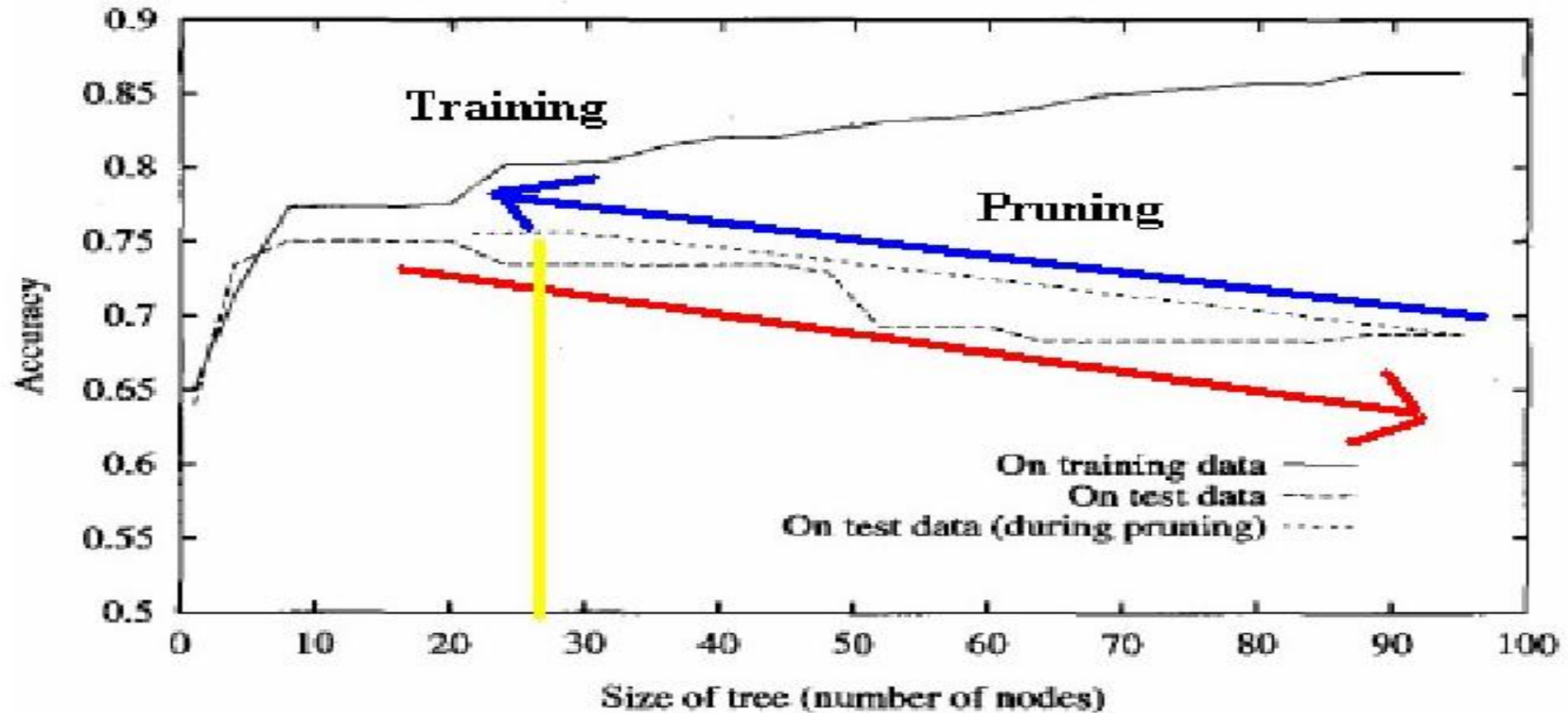
- Approach-1: (pre-pruning) Stop growing the tree earlier, before it reaches the point where it perfectly classifies the training data
 - Direct approach
 - Difficult to estimate precisely when to stop growing the tree.
- Approach-2: (post-pruning) Allow the tree to overfit the data, and then post-prune the tree – found to be more successful in practice
 - Reduced-error pruning (Quinlan 1987)
 - Rule post-pruning (Quinlan 1993)

Reduced Error Pruning

- **Pruning a decision node:**
 - Removing the subtree rooted at that node
 - Make it a leaf node
 - Assigning it the **most common classification** of the training examples affiliated with that node.
- Nodes are removed only if the resulting pruned tree performs **no worse than the original over the validation set**.



Reduced Error Pruning



Rule Post-Pruning

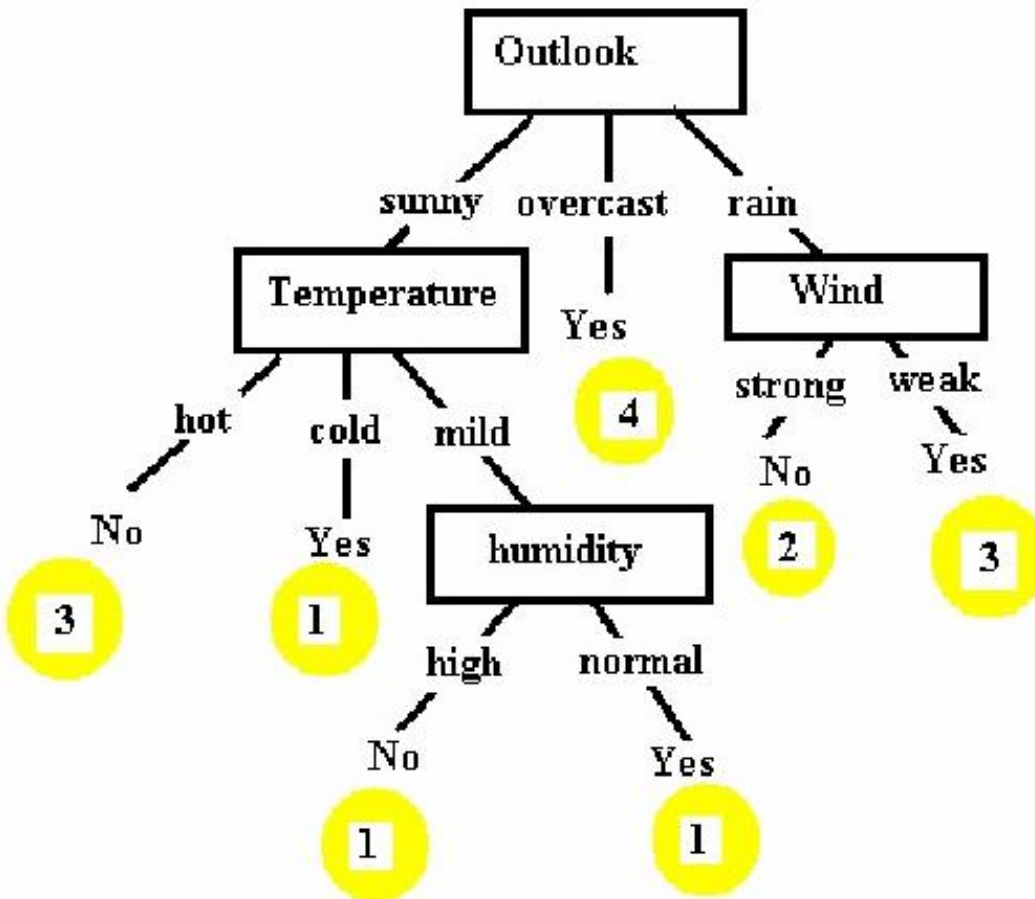
In practice, it is one quite successful method for finding high accuracy hypotheses in post-pruning of decision tree.

Rule Post-Pruning

1. Infer the decision tree from the training set, growing the tree until the training data is fit as well as possible and allowing overfitting to occur.
2. Convert the learned tree into an equivalent set of rules by creating one rule for each path from the root node to a leaf node.
3. Prune (generalize) each rule by removing any preconditions that result in improving its estimated accuracy.
4. Sort the pruned rules by their estimated accuracy, and consider them in this sequence when classifying subsequent instances.

Rule Post-Pruning (Step 1)

1. Infer the decision tree from the training set, growing the tree until the training data is fit as well as possible and allowing overfitting to occur.

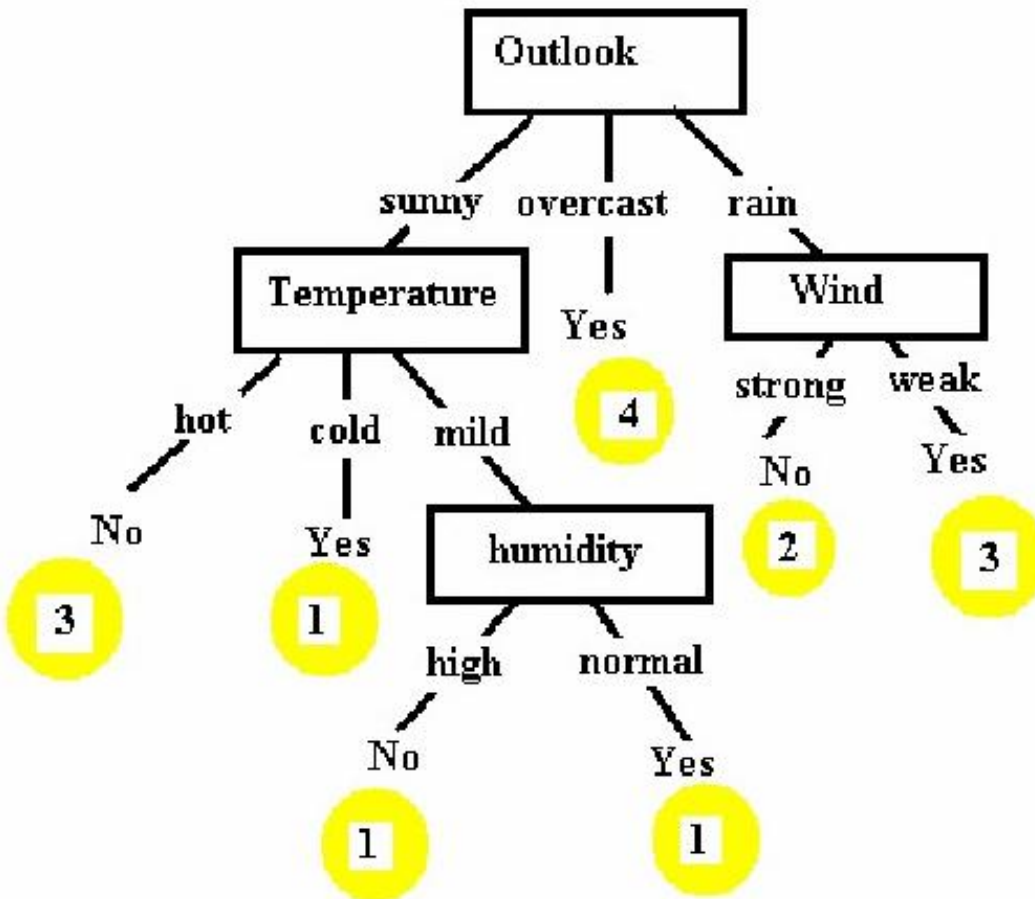


Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

D15 Sunny Hot Normal Strong No

Rule Post-Pruning (Step 2)

- 1: IF (Outlook = sunny and Temperature = Hot) THEN PlayTennis = No
- 2: IF (Outlook = sunny and Temperature = Cold) THEN PlayTennis = Yes
- 3: IF (Outlook = sunny and Temperature = Mild and Humidity=High) THEN PlayTennis = No
- 4: IF (Outlook = sunny and Temperature = Mild and Humidity=Normal) THEN PlayTennis = Yes
- 5: IF (Outlook = overcast) THEN PlayTennis = Yes
- 6: IF (Outlook = rain and Wind = Strong) THEN PlayTennis = No
- 7: IF (Outlook = rain and Wind = Weak) THEN PlayTennis = Yes



Rule Post-Pruning (Step 2)

2

Convert the learned tree into an equivalent set of rules by creating one rule for each path from the root node to a leaf node.

- 1: **IF** (Outlook = sunny and Temperature = Hot) **THEN** PlayTennis = No
- 2: **IF** (Outlook = sunny and Temperature = Cold) **THEN** PlayTennis = Yes
- 3: **IF** (Outlook = sunny and Temperature = Mild and Humidity=High) **THEN** PlayTennis = No
- 4: **IF** (Outlook = sunny and Temperature = Mild and Humidity=Normal) **THEN** PlayTennis = Yes
- 5: **IF** (Outlook = overcast) **THEN** PlayTennis = Yes
- 6: **IF** (Outlook = rain and Wind = Strong) **THEN** PlayTennis = No
- 7: **IF** (Outlook = rain and Wind = Weak) **THEN** PlayTennis = Yes

Rule Post-Pruning (Step 3)

3

Prune (generalize) each rule by removing any preconditions that result in improving its estimated accuracy.

1: IF (Outlook = sunny and Temperature = Hot) THEN PlayTennis = No

IF (Outlook = sunny and Temperature = Hot) THEN PlayTennis = No

IF (Outlook = sunny) THEN PlayTennis = No

IF (Temperature = Hot) THEN PlayTennis = No

(Validation
examples)

Rule Post-Pruning (Step 3)

3

Prune (generalize) each rule by removing any preconditions that result in improving its estimated accuracy.

IF (Outlook = sunny and Temperature = Hot) THEN PlayTennis = No

Acc1 = 85%

IF (Outlook = sunny) THEN PlayTennis = No

Acc2 = 89%

IF (Temperature = Hot) THEN PlayTennis = No

Acc3 = 77%

(Validation
examples)

If Acc2 > Acc3 & Acc1

1: IF (Outlook = sunny and Temperature = Hot) THEN PlayTennis = No

IF (Outlook = sunny) THEN PlayTennis = No

Rule Post-Pruning (Step 4)

4

Sort the pruned rules by their estimated accuracy, and consider them in this sequence when classifying subsequent instances.

R1: Acc4
R2: Acc3
R3: Acc2
R4: Acc1

Sort rules in descending order
of their accuracy on test
dataset or validation examples

S1: Acc1
S2: Acc2
S3: Acc3
S4: Acc4

R11: Acc14
R12: Acc13
R13: Acc12
R14: Acc11

S11: Acc11
S12: Acc12
S13: Acc13
S14: Acc14

S1: Acc1 >= S2: Acc2 >= S3: Acc3 >= S4: Acc4 >= ... >= S11: Acc11 >= S12: Acc12 >= S13: Acc13 >= S14: Acc14

Issues in Decision Tree Learning

- Determining how deeply to grow the decision tree
- Handling **continuous attributes**
- Choosing an appropriate attribute selection measure
- Handling training data with missing attribute values
- Handling attributes with differing costs

Handling Continuous-Valued Attribute

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Handling Continuous-Valued Attribute

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

90

Continuous values

Temperature:	40	48	60	72	80	90
PlayTennis:	No	No	Yes	Yes	Yes	No

Handling Continuous-Valued Attribute

<i>Temperature:</i>	40	48	60	72	80	90
<i>PlayTennis:</i>	No	No	Yes	Yes	Yes	No

We have dynamically **defined new discrete valued attributes** so that it partition the continuous attribute value into a discrete set of intervals.

<i>Temperature:</i>	40	48	60	72	80	90
<i>PlayTennis:</i>	No	No	Yes	Yes	Yes	No

<i>Temperature:</i>	40	48	60	72	80	90
<i>PlayTennis:</i>	No	No	Yes	Yes	Yes	No

Issues in Decision Tree Learning

- Determining how deeply to grow the decision tree
- Handling continuous attributes
- Choosing an appropriate **attribute selection measure**
- Handling training data with missing attribute values
- Handling attributes with differing costs, and improving computational efficiency

Alternative Measures for Selecting Attributes

- We use **information gain** measure to select attributes as root
- Consider the attribute **Day**, which has a very **large number of possible values**.
- If it would be selected as the decision attribute for the **root node** of the tree and lead to a **(quite broad) tree of depth one**, which perfectly classifies the training data.
- However, **this decision tree would classify poorly** on subsequent examples, because **it is not a useful predictor** – overfit.

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Alternative Measures for Selecting Attributes

- One way to avoid this difficulty is to select decision attributes based on **some measure other than information gain**.
- One alternative measure that has been used successfully is the **gain ratio** (Quinlan 1986).
- The gain ratio measure uses **split information**, that is sensitive to how **broadly and uniformly the attribute splits the data**.

$$SplitInformation(S, A) \equiv - \sum_{i=1}^c \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|}$$

$$GainRatio(S, A) \equiv \frac{Gain(S, A)}{SplitInformation(S, A)}$$

$$\underline{Gain(S, Outlook) = 0.246}$$

$$Gain(S, Humidity) = 0.151$$

$$Gain(S, Wind) = 0.048$$

$$Gain(S, Temperature) = 0.029$$

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

$$\text{Gain Ratio (S, OL)} = \mathbf{0.1559}$$

$$\text{Gain Ratio (S, Temp)} = 0.01863$$

$$\text{Gain Ratio (S, Hum)} = 0.1475$$

$$\text{Gain Ratio (S, Wind)} = 0.0487$$

Find the Gain Ratio (S, Day) and decide the root element.

Issues in Decision Tree Learning

- Determining how deeply to grow the decision tree
- Handling continuous attributes
- Choosing an appropriate attribute selection measure
- Handling **training data with missing attribute values**
- Handling attributes with differing costs

Most
repeated/
occurred

Mean
Average

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny		Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Issues in Decision Tree Learning

- Determining how deeply to grow the decision tree
- Handling continuous attributes
- Choosing an appropriate attribute selection measure
- Handling training data with missing attribute values
- Handling **attributes with differing costs**

Handling Attributes with Different Cost

- In some learning tasks, the instance attributes may have associated costs.
- For example, in learning to classify medical diseases
 - we might describe patients in terms of attributes such as **Temperature**, **BiopsyResult**, **Pulse**, **BloodTestResults**, etc.
- These attributes vary significantly in their costs, both in terms of monetary cost and cost to patient comfort.
- In such tasks, we would prefer **decision trees that use low-cost attributes where possible**, relying on high-cost attributes only when needed to produce reliable classifications.

$$\text{select attribute based on cost} \rightarrow \frac{\text{Gain}(S, A)}{\text{Cost}(A)}$$

Summary of issues in Decision Tree Learning

- Determining how deeply to grow the decision tree – pruning methods
- Handling continuous attributes - defining new discrete valued attributes
- Choosing an appropriate attribute selection measure – Gain ratio
- Handling training data with missing attribute values – most common
- Handling attributes with differing costs – consider cost factor of the attribute

The most notable types of decision tree algorithms are:-

1. **Iterative Dichotomiser3 (ID3)**: This algorithm uses **Information Gain** to decide which attribute is to be used classify the current subset of the data. For each level of the tree, information gain is calculated for the remaining data recursively.
2. **C4.5**: This algorithm is the successor of the ID3 algorithm. This algorithm uses either **Information gain or Gain ratio** to decide upon the classifying attribute. It is a direct improvement from the ID3 algorithm as it can **handle both continuous and missing attribute values**.
3. **Classification and Regression Tree (CART)**: It is a dynamic learning algorithm which can produce a **regression tree** as well as a **classification tree** depending upon the **dependent variable**. Gini

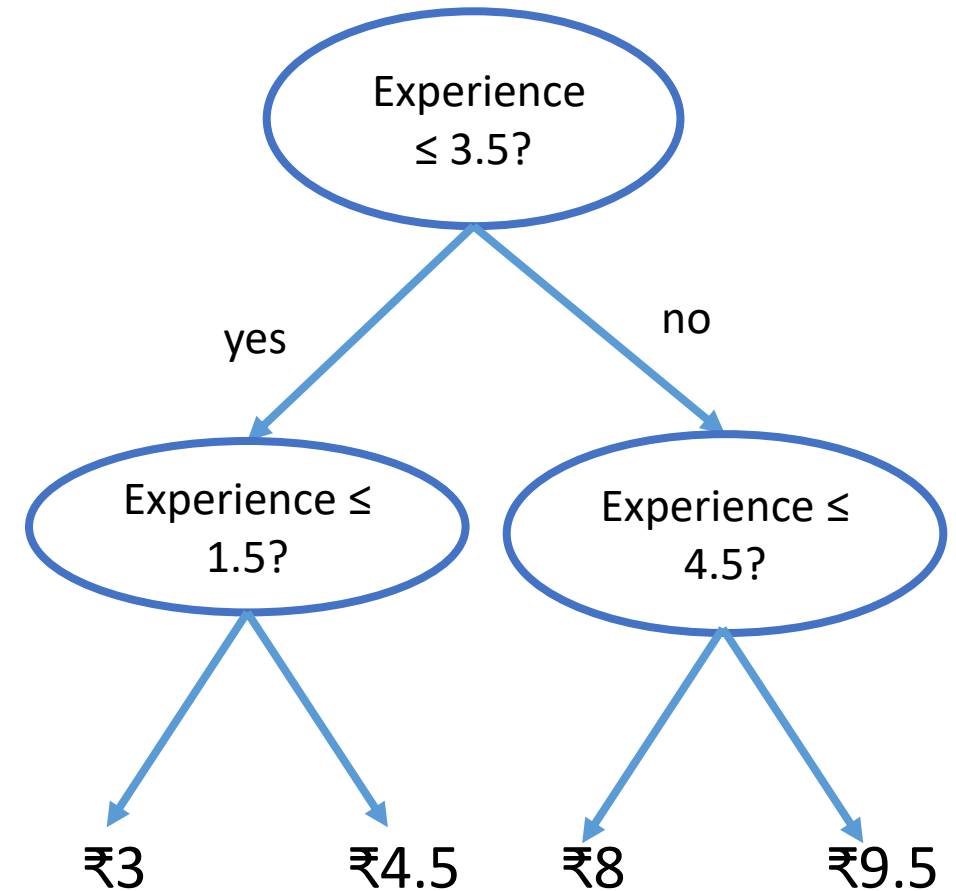
Decision Tree Regressor



Decision Tree Regressor

- A **Decision Tree Regressor** is used when the target/output variable is **continuous numerical data**, such as salary, house price, temperature, sales, or marks.

Experience (Years)	Salary (₹ lakh)
1	3
2	4
3	5
4	8
5	9
6	10



But How Does the Tree Choose a Split?

- For regression, we need to measure **how spread out the numerical target values are**.
- A common criterion is **Mean Squared Error (MSE)**.
- For a node containing n observations:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$$

- The tree searches for a split that **reduces squared error as much as possible**.

Example

Experience	Salary
1	3
2	4
3	5
4	8

Consider the possible split:
Experience ≤ 2.5
This creates:

Left Node

Salary:

3,4

Mean = 3.5

MSE = 0.25

$$\bar{y}_L = \frac{3 + 4}{2} = 3.5$$

$$MSE_L = \frac{(3 - 3.5)^2 + (4 - 3.5)^2}{2}$$

$$MSE_L = 0.25$$

Right node:

Salary:

5,8

Mean:

$$\bar{y}_R = \frac{5 + 8}{2} = 6.5$$

$$\begin{aligned}MSE_R &= \frac{(5 - 6.5)^2 + (8 - 6.5)^2}{2} \\&= \frac{2.25 + 2.25}{2}\end{aligned}$$

$$MSE_R = 2.25$$

The Decision Tree calculates this for candidate splits and generally prefers the split producing the **lowest weighted impurity/error**.

Weighted MSE

Because both nodes contain two observations:

$$\begin{aligned}MSE_{split} &= \frac{2}{4}(0.25) + \frac{2}{4}(2.25) \\&= 0.125 + 1.125\end{aligned}$$

$$MSE_{split} = 1.25$$

Compare Another Split
Experience ≤ 3.5

Left:

3,4,5

Mean: $\bar{y}_L = 4$

$$MSE_L = \frac{(3 - 4)^2 + (4 - 4)^2 + (5 - 4)^2}{3}$$

$$= \frac{1 + 0 + 1}{3} = 0.667$$

Right:

8

Since there is only one value:

$$MSE_R = 0$$

Weighted MSE:

$$MSE_{split} = \frac{3}{4}(0.667) + \frac{1}{4}(0)$$

$$MSE_{split} \approx 0.50$$

Split	Weighted MSE
Experience ≤ 2.5	1.25
Experience ≤ 3.5	0.50

Therefore, between these two candidates:

Experience ≤ 3.5 is the better split.

This is the core idea behind Decision Tree Regression.

Prediction:

Suppose after training, a leaf contains salaries:
8,9,10

The leaf prediction is:

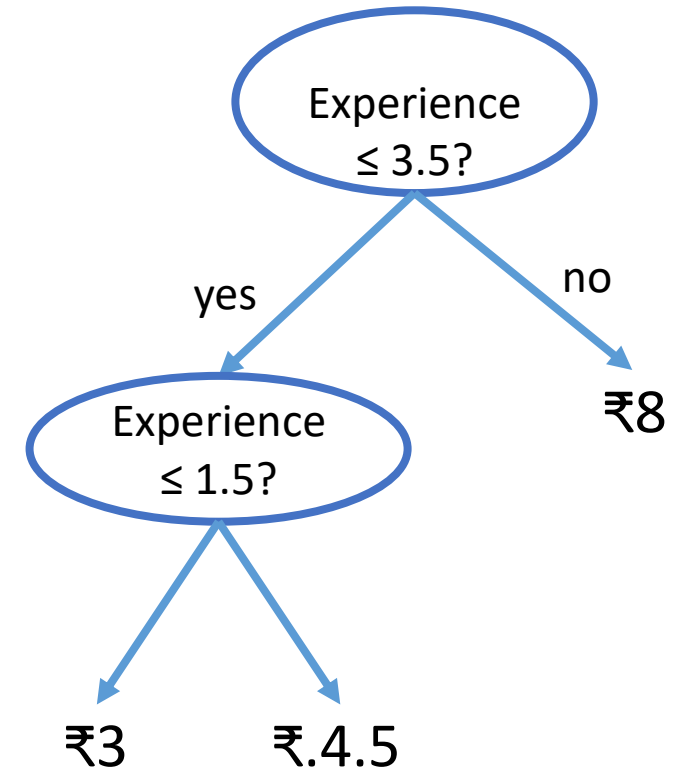
$$\hat{y} = \frac{8 + 9 + 10}{3}$$

$$\hat{y} = 9$$

So a new employee reaching this leaf would get a predicted salary of approximately:

₹9 lakh

And the main numerical idea is:
Choose splits that minimize prediction error within the resulting nodes.



Find the best split and then recursively construct the tree for the following dataset

Employee	Experience (X)	Salary (Y) in ₹ lakh
A	1	2
B	2	3
C	3	5
D	4	8
E	5	10

Summary

What is DT

Uses

Types

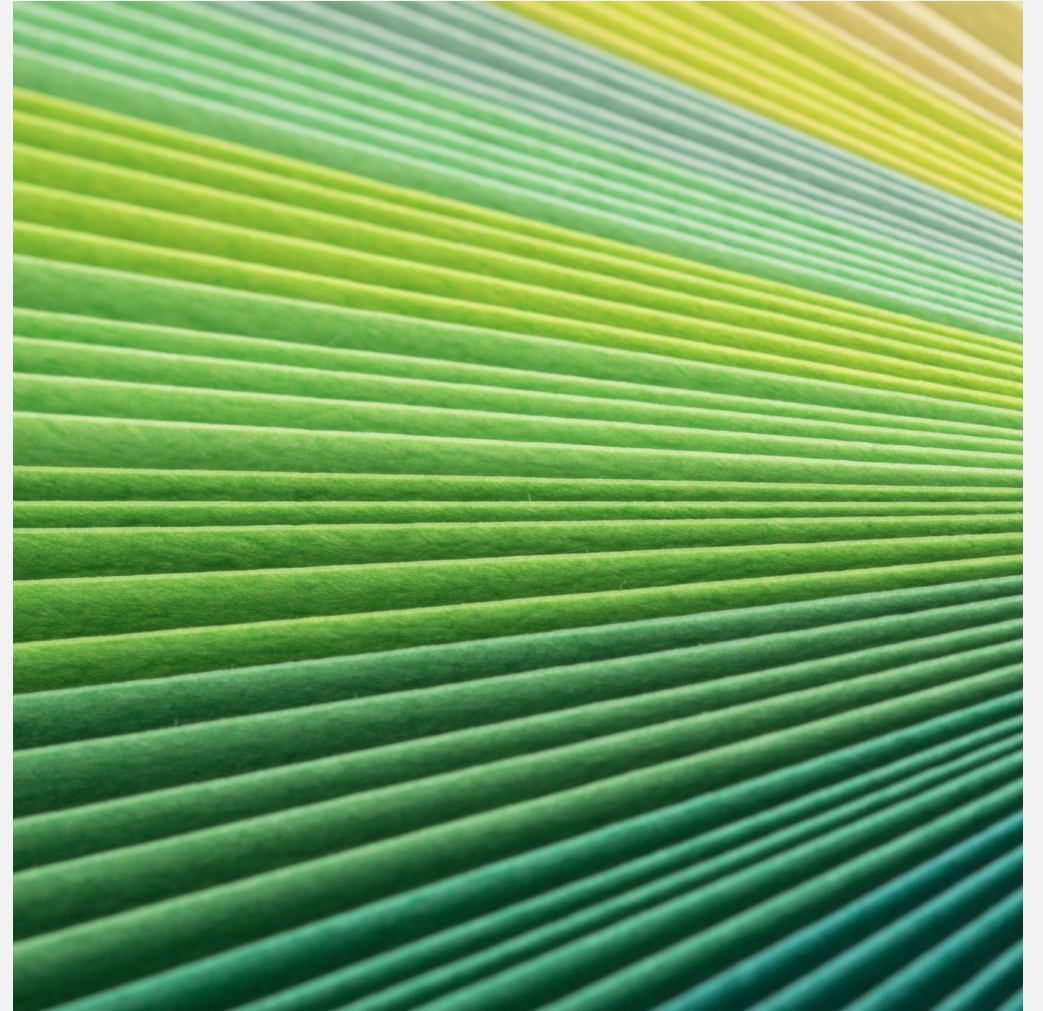
DT based on information gain

DT based on Gini

Issues in DT and methods to address them

Training-test-validation

Overfitting in DT



Thank you